

COSMIC–ASCEC

Automated Configurational Sampling and Topological Screening of Molecular Clusters

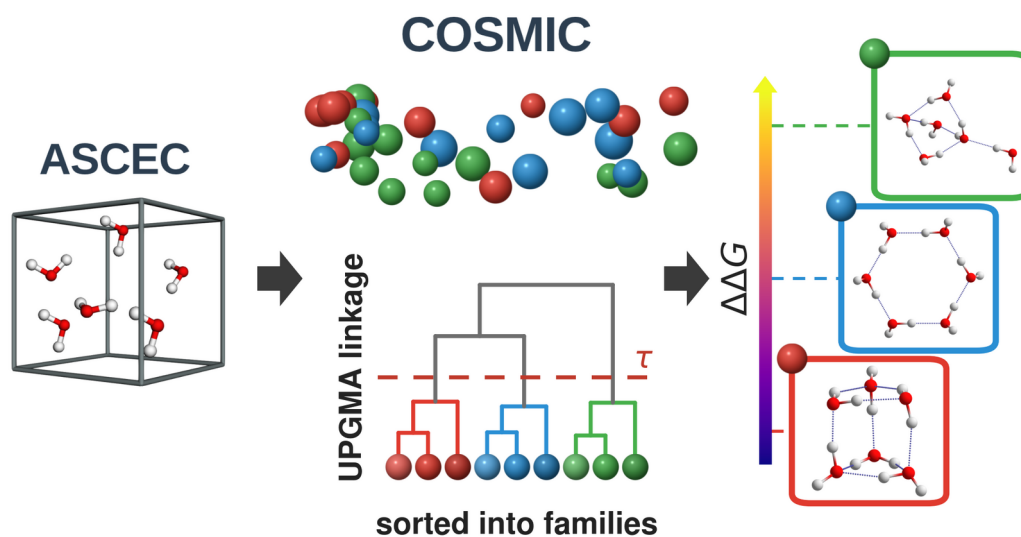
User Manual

Manuel Gómez[†] – Sara Gómez[‡] – Albeiro Restrepo[†]

Química Física Teórica

[†]*Instituto de Química, Universidad de Antioquia UdeA, Medellín, Colombia*

[‡]*Universidad Nacional de Colombia, Departamento de Química, Bogotá, Colombia*



Last Updated: June 2026

Contents

1	Distribution	5
2	Introduction	5
2.1	What is ASCEC?	5
2.2	How It Works (Overview)	5
3	Theoretical Background	6
3.1	Simulated Annealing Algorithm	6
3.2	Monte Carlo Moves	7
3.3	Topological Clustering (COSMIC)	7
3.3.1	The Physicochemical Feature Vector	8
3.3.4	Quality Control	9
4	Installation	9
4.1	System Recommendations	9
4.2	Required Software	9
4.2.1	Core Python Dependencies	9
4.2.2	Quantum Chemistry Backends	10
4.3	Step-by-Step Installation	10
4.3.1	Automatic Installation	10
4.3.2	Conda Environment	11
5	Web Input Generator	14
5.1	Features	14
5.2	Accessing the Tool	14
5.3	GUI Modes	14
5.4	Interface Overview	15
6	Quick Start: A Black-Box Example	15
6.1	Glycolaldehyde-Water	15
7	Water Hexamer	20
7.1	Molecular Coordinates Generation	20
7.2	Mode Selection	20
7.3	Simulation Box	21
7.4	Annealing Parameters	22
7.5	Geometry optimization and refinement	22
7.6	Energy Refinement & COSMIC Screening	23
7.7	Pipeline Review and Execution	23
8	Advanced Usage & Reference	29
9	Input File Format	29
9.1	Part 1: Simulation Parameters & Coordinates	29
9.2	Part 2: The Automated Protocol & Embedded Templates	30
10	Box Size Suggestion	32
10.5	Choosing the Packing Fraction	34
11	Understanding Each Stage	35
11.1	Annealing and the Box	35
11.2	Geometry Optimization	36

11.2.1	Input Templates	36
11.2.2	Running and Sorting	37
11.3	COSMIC Clustering	38
11.3.1	Configuration Screening	38
11.3.2	Feature Vector Technicalities	38
11.3.3	A Dynamic Feature Vector	39
11.3.4	Automatic Threshold Selection	39
11.3.5	Running COSMIC	41
11.4	Refinement of the Motifs	41
12	Output Files	42
12.1	Print Levels: miniprint and maxprint	42
12.2	Annealing output	42
12.2.1	Main Log File (w6.out)	42
12.2.2	Reproducibility and Metadata	43
12.2.3	Simulation Generated Data	44
12.3	Calculation & Optimization Output	47
12.4	COSMIC Output	47
12.4.1	Dendrograms	48
12.4.2	Clustering Summary Log	52
12.4.3	Error Handling and Diagnostics	53
12.4.4	Generated Cluster Data	53
12.4.5	Boltzmann Distribution	54
13	Workflow Mode	55
13.1	Pipeline Logic	55
13.2	Workflow Input (formic.asc)	55
13.3	Protocol Command Syntax	58
13.4	Execution and Outputs	58
13.4.1	Prerequisites and File Structure	58
13.4.2	Terminal Output	59
13.5	Execution Control and Resumption	59
13.6	Failure Recovery Mechanisms	60
13.6.1	Retry Logic (Execution Reliability)	60
13.6.2	Redo Logic (Convergence and Topology)	61
14	Results and Validation	62
14.1	Standalone Execution: Water Hexamer	62
14.1.1	Hierarchical Two-Step Protocol (ORCA)	62
14.1.2	Direct Approach (Gaussian 09) & Comparative Analysis	65
14.2	Automated Workflow: Formic Acid Dimer	66
14.2.1	Protocol and Computational Efficiency	67
14.2.2	Automated Error Correction and Robustness	67
14.2.3	Thermodynamic Analysis	68
14.3	Clustering Methodologies: Topology vs. Geometry	70
14.3.1	Case Study A: Resolving Spatial Isomerism (Adenine-Thymine)	70
14.3.2	Case Study B: Handling Symmetry and Indexing (Arsenate Anion)	72
14.3.3	Clustering Overview	73
15	Command Reference	73
16	Quick ORCA 6.1.1 Installation for Linux	75

17 AI Acknowledgement and Reproducibility	78
18 References	78

1 Distribution

The ASCEC-v04 & COSMIC-v01 package is distributed via a public GitHub repository:

- **Repository:** <https://github.com/manuel2gl/qft-cosmic-ascec>
- **License:** GNU General Public License (see LICENSE file for details)
- **Supported OS:** Primarily developed and tested on Linux (Ubuntu, CentOS, Fedora). Preliminary runs are also tested on Windows using xTB 6.7.1. macOS should work with minor adjustments.
- **Contents:**
 - Python scripts for configurational / conformational sampling, clustering, and analysis
 - Documentation and brief theoretical description
 - Example input files and workflows
 - Web based input generator:
 - * <https://manuel2gl.github.io/qft-cosmic-ascec/>
- **Requirements:**
 - Python 3.9 or higher (3.11 suggested)
 - ORCA or xTB for quantum chemistry calculations

2 Introduction

2.1 What is ASCEC?

ASCEC (Annealing Simulado Con Energía Cuántica / Simulated Annealing with Quantum Energy) is a computational tool for automated configurational sampling and analysis of molecular clusters. It integrates:

- **Simulated Annealing:** Efficient exploration of potential energy surfaces
- **Hierarchical Clustering (COSMIC):** Identification of representative structures using a physicochemical feature vector.

2.2 How It Works (Overview)

The ASCEC pipeline operates in three stages:

1. **Sampling (Annealing):** A simulated annealing algorithm randomly moves, rotates, and deforms the molecules inside a box. At each step, a quantum chemistry package (ORCA or xTB) evaluates the energy. The algorithm gradually “cools down”, favouring low energy structures while still allowing occasional uphill moves to escape local traps. This produces hundreds of candidate configurations.
2. **Clustering (COSMIC):** Most of these candidates are redundant; to overcome this limitation, the COSMIC (COnfigurational Similarity via Motif Identification Clustering) module compares them using a fingerprint constructed from their energies, orbital vacancies, dipole moments, and other properties. Structures with similar fingerprints are grouped into families. Only one representative per family (a **motif**) is retained for further study.
3. **Refinement:** The motifs are refined at a higher level of theory (e.g., DFT) to obtain publication quality geometries, energies, and thermodynamic populations.

3 Theoretical Background

3.1 Simulated Annealing Algorithm

The ASCEC module drives the exploration of the configurational space through a Markov Chain Monte Carlo (MCMC) protocol implemented within a simulated annealing framework [1]. To ensure statistical significance, the algorithm generates thousands of trial configurations. The quantum energy of every generated structure is calculated using efficient methods (supporting any theoretical level available in the external package, e.g., GFN2-xTB, PM3, AM1, HF).

The acceptance of a structural perturbation depends on the change in potential energy, $\Delta E = E_{new} - E_{old}$. If the structural change lowers the energy ($\Delta E < 0$), the move is accepted immediately. In the case of an energy increase ($\Delta E > 0$), the acceptance behavior depends on the chosen criterion:

Standard Metropolis Criterion [2] (Optional) When explicitly requested (via `--standard` flag), the algorithm follows the classical Boltzmann probability. An uphill move is accepted if a random number $\xi \in [0, 1]$ satisfies:

$$\xi < e^{-\frac{\Delta E}{k_B T}} \quad (1)$$

Modified Metropolis Criterion (Default) To mitigate kinetic trapping in local basins and prevent jumps to physically unrealistic high-energy configurations ASCEC implements a modified acceptance condition [3]. Unlike the standard probabilistic approach, which compares the Boltzmann factor to a random number, this method evaluates the relative energy cost of the perturbation. The new structure is accepted if:

$$\Phi(\Delta E) < P(\Delta E) \quad (2)$$

where $P(\Delta E) = e^{-\Delta E/k_B T}$ is the temperature dependent Boltzmann probability, and $\Phi(\Delta E)$ represents the relative energy change defined as:

$$\Phi(\Delta E) = \frac{\Delta E}{|E_{new}|} \quad (3)$$

This criterion was found to be more adequate for this class of problems. By removing the stochastic element found in the standard Metropolis method, the algorithm avoids the accidental rejection of chemically viable structures due to random number generation, while still filtering out moves where the relative energy increase is drastic.

Temperature Schedules: The simulation temperature decreases over n_T steps to gradually reduce the acceptance of high-energy states. A choice between a linear (decreasing T by a constant amount) and a geometric (decreasing T by a constant percentage) route is available:

- *Linear Quenching:* The temperature is reduced by a fixed amount dT at each step:

$$T_{n+1} = T_n - dT$$

- *Geometric Quenching:* The temperature is scaled down by a cooling factor f (in %):

$$T_{n+1} = T_n \times \left(1 - \frac{f}{100}\right)$$

3.2 Monte Carlo Moves

The configuration space is explored using three perturbation motions:

1. **Translation:** Random displacement of the center-of-mass.
2. **Rotation:** Random rotation around principal axes.
3. **Conformational (Optional):** Modification of internal dihedral angles.

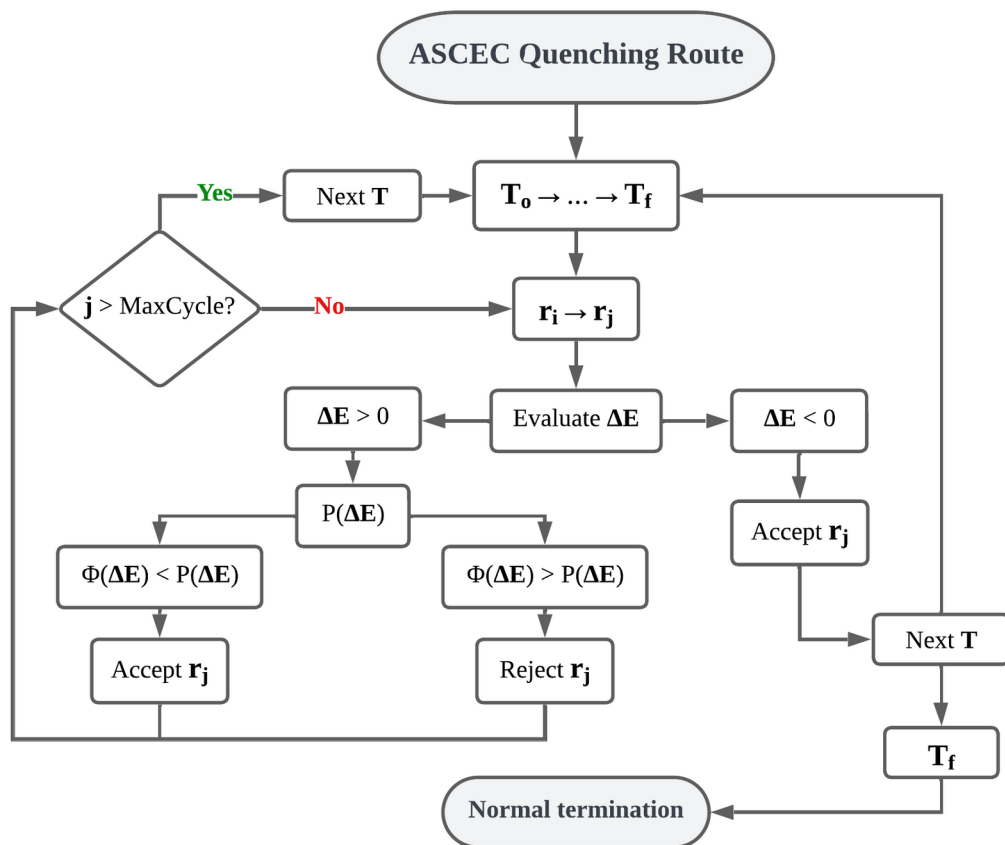


Figure 1: SA procedure using a modified Metropolis criterion.

3.3 Topological Clustering (COSMIC)

While stochastic exploration (ASCEC) is effective at traversing energy barriers, it inherently suffers from a data management limitation: the generation of highly redundant datasets. A typical annealing run yields hundreds of candidate structures, the vast majority of which converge to identical basins of attraction. To address this, the **COSMIC** module functions as a post-processing discriminator designed to filter, validate, and cluster the raw output into unique chemical motifs.

The scope of this algorithm is to bridge the gap between stochastic chaos and ordered chemical insight. By reducing massive datasets into non-redundant families, it facilitates the manual revision of the Potential Energy Surface (PES) and ensures that only topologically distinct minima are subjected to high-level quantum mechanical refinement.

3.3.1 The Physicochemical Feature Vector

Standard clustering methods often rely solely on geometric superposition (RMSD), which depends on atomic indexing [4]. To overcome this, we define the configurational identity of a structure i using a multi-dimensional scalar feature vector, $\mathbf{x}_i$. This vector aggregates continuous physicochemical descriptors derived from the quantum mechanical data. The **full vector** (up to 15 dimensions) is:

$$\mathbf{x}_i = (E_{elec}, V_{nuc}, G, E_{HOMO}, \Delta E_{HL}, \mu, A, B, C, \nu_{smallest}, \nu_{largest}, n_{HB}, \overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}) \quad (4)$$

where the components represent energetics (E_{elec} , G); V_{nuc} : nuclear repulsion energy; E_{HOMO} : energy of the HOMO; ΔE_{HL} : HOMO–LUMO gap; μ : dipole moment; A, B, C : rotational constants; $\nu_{smallest}, \nu_{largest}$: smallest and largest vibrational frequencies; n_{HB} : number of hydrogen bonds, and hydrogen bonding descriptors $\overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}$: geometrical parameters defining hydrogen bonds within each structure.

All structures are clustered together in a single pool by default no pre-grouping by hydrogen-bond count is performed. H-bond information is encoded indirectly through $\overline{d_{HB}}$ and $\overline{\theta_{HB}}$, allowing the clustering algorithm to account for H-bond patterns without imposing rigid partitions. The optional `--group-hb` flag restores per H-bond count grouping.

3.3.2 Weighted COSMIC Metric

Since the vector components possess disparate physical units, a feature-wise Z-score standardization is applied $z_i^k = (\gamma_i^k - \bar{\gamma}^k / \sigma^k)$, $Z_i^k = w^k \times z_i^k$. The distance between two structures is then quantified using the Euclidean distance:

$$d(\mathbf{X}'_i, \mathbf{X}'_j) = \sqrt{(\mathbf{X}'_i - \mathbf{X}'_j) \cdot (\mathbf{X}'_i - \mathbf{X}'_j)} = \sqrt{\sum_{k=1}^{n_d} (Z_i^k - Z_j^k)^2} \quad (5)$$

where w^k represents user-definable weights used to bias the clustering focus (e.g., prioritizing H-bonding patterns over soft vibrational modes).

3.3.3 Hierarchical Two-Step Clustering

To ensure both computational efficiency and high structural fidelity, the clustering process operates via a hierarchical "funnel" protocol involving two distinct steps:

1. **Step I: Physicochemical Clustering (Coarse-Grained).**

The algorithm first performs agglomerative hierarchical clustering using UPGMA (average linkage) based on the z -score-standardised feature vectors $\mathbf{x}$. This rapidly groups configurations that share similar energetics, electronic states, and general shapes into broad families. The dendrogram is cut at a user-defined distance threshold τ to obtain a flat partition. This step drastically reduces the dataset size by merging redundant snapshots without performing expensive geometric alignments.

2. **Step II: Optional RMSD Refinement (Fine-Grained).**

Optionally, a secondary geometric check is applied within each identified cluster. A Root Mean Square Deviation (RMSD) analysis [5] is performed to distinguish stereoisomers or iso-energetic conformers that map to identical scalar vectors (e.g., enantiomers or distinct proton ordering networks with identical energies). This ensures that subtle geometric variations are not lost during the vector-based reduction.

3.3.4 Quality Control

Beyond clustering, the module serves as a topological filter. It screens for the presence of **one or more imaginary frequencies**, indiscriminately identifying first-order saddle points (transition states) and higher-order instabilities. If such a structure clusters tightly with a true minimum, it is discarded as a redundant, non-converged distortion. However, if it forms an isolated cluster, it is flagged as a unique topological feature and output for re-optimization, ensuring that potential basins are not overlooked merely due to incomplete convergence.

4 Installation

4.1 System Recommendations

Although Python scripts are lightweight, quantum chemistry backends are resource-intensive:

- **RAM:** Minimum 8GB (16GB+ recommended for DFT calculations).
- **CPU:** Multi-core processor (ASCEC can parallelize ORCA jobs).
- **Storage:** SSD recommended. Optimization trajectories can generate large text files.
- **OS:** Linux (Ubuntu/CentOS) is the primary supported environment. Windows is supported but may require path adjustments.

4.2 Required Software

4.2.1 Core Python Dependencies

The core functionality is based on standard Python scientific libraries and other software.

Package	Purpose
NumPy	High-performance array operations and linear algebra.
SciPy	Optimization routines and statistical functions.
scikit-learn	Algorithms for hierarchical clustering and data preprocessing.
Matplotlib	Generation of energy plots and dendrograms.
cclib	Parsing of legacy ORCA ($\leq 5.0.x$) logs.
OPI	Parsing of modern ORCA (≥ 6.1) outputs.
OpenBabel	File generation and format conversion.
xTB	Highly efficient semiempirical quantum chemical framework.

Table 1: Python libraries required for ASCEC and COSMIC modules.

***Compatibility Note:** The **COSMIC** module automatically selects the appropriate parser based on the output file version. **cclib** is used for ORCA $\leq 5.0.x$, while **OPI** is required for ORCA 6.1+. *Note: ORCA 6.0 is not supported by either parser; please use v5.0.x or upgrade to v6.1+.* At least one parser must be installed.

4.2.2 Quantum Chemistry Backends

The workflow requires an external electronic structure package to perform QM calculations.

ORCA: ORCA [6] is free for academic use and supports a broad range of high-level ab initio and DFT methods. It is fully integrated with both the ASCEC and COSMIC modules. See Section 16 for installation instructions.

xTB: xTB [7] is the default backend for both the annealing and preliminary preoptimization stages due to its speed. It is installed automatically from `conda-forge` by `install.sh` and `win_install.bat`.

4.3 Step-by-Step Installation

4.3.1 Automatic Installation

We provide a simplified shell script that automates the entire process. It installs Miniconda (if not present), sets up the dependencies (xTB, OpenBabel, cclib, OPI), and configures the necessary aliases. The aliases point directly to the environment's Python binary, so no manual `conda activate` is needed to run `ascec` or `cosmic`. First download the `install.sh` script.

```
wget https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/install.sh
```

Installation Steps

The installation can be performed by executing the `install.sh` script, which automatically uses a Python 3.11 environment (py11).

```
bash install.sh
```

```
source ~/.bashrc
```

Alternatively, if the `base` environment is preferred, install the latest Miniconda and downgrade the Python version within `base`, then run the script again.

```
micro install.sh
```

```
#=====
# CONFIGURATION
#=====
# Set to TRUE to create a separate 'py11' environment with Python 3.11
# Set to FALSE to install into the base conda environment (default)

INSTALL_PY11=TRUE
```

Save the file (Ctrl+S) and exit (Ctrl+Q).

Uninstall (Linux)

To reverse the installation without touching your Conda base, run the provided uninstaller.

```
bash uninstall.sh
```

This removes the `py11` environment, the `~/software/ascec04` directory, and the `ascec/cosmic` aliases from `.bashrc`. Conda itself is left intact.

Windows Installation

On Windows, the same one-click workflow is provided as a `.bat` file (a hybrid batch/PowerShell script). No prior setup is required — simply double-click `win_install.bat`, or run it from a Command Prompt. The script installs Miniconda if absent, creates the `py11` environment, installs all dependencies (including xTB 6.7.1), and registers `ascec` and `cosmic` as system-wide launchers. If Windows blocks execution, right-click the file and select *Run as administrator*.

```
wget https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/win_install.bat
```

```
win_install.bat
```

Uninstall (Windows)

Double-click `win_uninstall.bat` (or run from Command Prompt) to remove the `py11` environment, the source directory, and the launchers. Miniconda is left untouched.

```
win_uninstall.bat
```

4.3.2 Conda Environment

I. Prepare the directory and clone the repository

Establish a dedicated directory for the software and clone the source repository. To synchronize an existing installation with the latest updates, execute `git pull`.

```
mkdir -p ~/software/ascec04
```

```
git clone https://github.com/manuel2gl/qft-cosmic-ascec.git ~/software/ascec04/
```

II. Install Miniconda (Skip if Conda is already installed)

Download and run the installer. During installation, type `yes` when asked to initialize conda.

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

```
bash Miniconda3-latest-Linux-x86_64.sh
```

III. Create and configure the environment

Create a clean Python 3.11 environment named `py11` (or your preferred name).

```
conda create -n py11 python=3.11 -y
```

IV. Set up Aliases

To make running the programs easier, add shortcuts to your shell configuration. By pointing directly to the environment's Python binary, no `conda activate` is needed at runtime. First, find your conda base path:

```
conda info --base
```

Open your `.bashrc` file:

```
micro ~/.bashrc
```

Scroll to the bottom of the file and paste the following lines, replacing `<conda_base>` with the path from the previous command (e.g., `/home/user/miniconda3`):

```
# COSMIC ASCEC aliases
alias ascec='<conda_base>/envs/py11/bin/python $HOME/software/ascec04/ascec-v04.py'
alias cosmic='<base>/envs/py11/bin/python $HOME/software/ascec04/cosmic-v01.py'
```

Save the file (Ctrl+S) and exit (Ctrl+Q). Then, reload the configuration:

```
source ~/.bashrc
```

V. Install Dependencies

Activate the environment and install the required libraries.

```
conda activate py11
```

```
conda install numpy scipy matplotlib scikit-learn -y
```

```
conda install -c conda-forge cclib openbabel -y
```

```
conda install -c conda-forge 'xtb>=6.7'
```

```
pip install orca-pi
```

Note: You can install both parsers for maximum compatibility with different ORCA versions. OPI is installed via pip even in conda environments.

VI. Verify Installation

Check that the commands are working (no environment activation needed):

```
python -c "import numpy, scipy, matplotlib, sklearn, cclib, opi; print('All packages OK')"
```

```
ascec --version
```

```
ascec --help
```

```
cosmic --help
```

```
orca -v
```

```
obabel -V
```

```
(base) manuel@unix:~$ ascec -h
```

```
usage: ascec [OPTIONS] COMMAND [ARGUMENTS]
```

ASCEC - Annealing Simulado con Energia Cuantica

(Simulated Annealing with Quantum Energy)

Configurational sampling via Monte Carlo with quantum mechanical evaluation

positional arguments:

COMMAND Input file or [command](#) (opt, ref, sort, cosmic, diagram, etc.)

ARG1 Command-specific argument (e.g., template file, mode)

ARG2 Additional [command](#)-specific argument

options:

-h, --help show this [help](#) message and [exit](#)

-v Increase verbosity level (use -v, -v2, -v3, etc.)

--standard Use standard Metropolis criterion instead of modified

--nosum Skip summary file generation during sort

--justsum Generate summary file only without sorting structures

--nobox Disable generation of box-visualization XYZ files

--maxprint Keep all intermediate files from every stage (default: miniprint, clean up at end)

-V, --version Display version information and [exit](#)

COMMANDS:

Run an annealing job (the COMMAND is the .asc input file itself):

ascec input.asc Run simulated annealing on input.asc

ascec input.asc rN Build N replicate runs (e.g. r3 = 3 replicas)

5 Web Input Generator

ASCEC-v04 includes a browser based tool to generate input files automatically, fetch structures from PubChem, and visualize the simulation box in 3D.

5.1 Features

- **PubChem Integration:** Automatically converts SMILES, IUPAC names (e.g., "aspirin"), or brand names into 3D XYZ coordinates using the PUG REST API [8].
Compounds search: <https://pubchem.ncbi.nlm.nih.gov/compound/>
- **3D Visualization:** Uses 3Dmol.js [9] to preview the simulation box, showing the boundaries and initial molecular placement.
- **Protocol Builder:** easy creation of multi-stage workflows (e.g., `opt`, `cosmic`, `ref`).

5.2 Accessing the Tool

You can access the generator in two ways:

1. **URL:** Visit <https://manuel2gl.github.io/qft-cosmic-ascec/>
2. **Direct:** Run the following command in your terminal:

```
ascec input
```

5.3 GUI Modes

The tool offers two complementary interfaces, selectable from the top navigation bar: **Smart Mode** (the default) and **Advanced Generation**. Both produce the same ASCEC input file; they differ only in how much of the configuration is exposed and how much is automated.

Smart Mode

Smart Mode is a guided, automated workflow designed to get a valid simulation running with minimal input. The typical sequence is:

1. **Build the system.** Enter any PubChem identifier (name, SMILES, CAS, or InChI, e.g. `water`, `64-17-5`, `CCO`) in the *Molecule* $\rightarrow$ *XYZ Converter* to fetch 3D coordinates, and set the number of copies for each species.
2. **Set the simulation box.** Specify the box length directly, or enable *auto* to compute it from a target packing percentage. The molecules and box boundaries are previewed live in the 3D viewer.
3. **Choose the parameters.** A guided form selects sensible defaults for the annealing, pre-optimization, refinement, and COSMIC settings, so you only adjust the key choices.
4. **Review and download.** A pipeline summary shows every stage of the workflow before you download the generated input file.

5.4 Interface Overview

The screenshot displays the COSMIC-ASCEC web interface. At the top, navigation links include 'Installation', 'Smart Mode' (selected), 'Advanced Generation', 'User Manual', and 'GitHub'. A header bar contains the text 'Automated configurational sampling and topological screening of molecular clusters', 'Química Física Teórica', and 'Universidad de Antioquia, Colombia'.

The main interface is divided into several sections:

- Simulation Box:** Features a 'Box Length (Å)' input set to 6.0, a '30 %' dropdown, and a checked 'auto' checkbox. A 'Dense' label is present. A 3D visualization shows a molecular cluster within a wireframe box. A 'Box Suggestion — adjust as needed' message is displayed. Below the visualization are buttons for 'Refresh', 'Visual Test', 'Download XYZ', 'Download Box XYZ', and an 'Enter simulation name' field.
- Molecule → XYZ Converter:** Includes a text input for 'Enter any PubChem identifier (name, SMILES, CAS, InChI) to generate 3D coordinates.' Below this is a 'Name, SMILES, CAS, or InChI' input field containing 'Diase', a 'Generate' button, and a grid of molecule buttons (Water, Ethanol, Formic Acid, Alanine, Benzene, Methane, Ammonia, Methanol, Acetone, Acetic Acid, Toluene, Phenol, Pyridine, THF, Hexane). There are also links for 'PubChem' and 'Novoprolabs 3D', a 'Download .mol from Novoprolabs and drop it on the preview.' instruction, and a 'Periodic Table' button.
- Molecule Preview:** Shows a 3D ball-and-stick model of a molecule. Below the model are buttons for '1', 'Add', 'XYZ', 'SMILES', 'Clear (3)', and a menu icon.
- Sampling Mode:** Contains three selectable options: 'Preliminary' (Stop after geometry optimization), 'Rigorous' (Stop after geometry refinement), and 'Ultimate' (Stop after energy refinement). A 'MAXPRINT' checkbox is located at the top right of this section.

At the bottom, a workflow progress bar shows the sequence: Annealing → Geometry Optimization → COSMIC 1 → Geometry Refinement → COSMIC 2 → Energy Refinement → COSMIC 3 → Results.

Figure 2: Web Input Generator interface with 3D visualization and parameter forms.

6 Quick Start: A Black-Box Example

6.1 Glycolaldehyde-Water

This section presents a minimal example that demonstrates the core functionality of COSMIC-ASCEC through the hydration of glycolaldehyde. It is intended as a black-box test that can be run without prior knowledge of the underlying theory or the input file syntax, providing a convenient way to verify the installation and obtain a first overview of the program's workflow and results.

The system consists of one glycolaldehyde and two water molecules. This is a particularly instructive test case because glycolaldehyde exhibits both **configurational** (intermolecular arrangement) and **conformational** (internal dihedral rotation) degrees of freedom, making it richer than a purely rigid-body system.

We begin by generating the initial coordinates with the web input generator. Glycolaldehyde is the simplest member of the diose family, so it can be located in the generator by searching for the name *diase* (Figure 3).

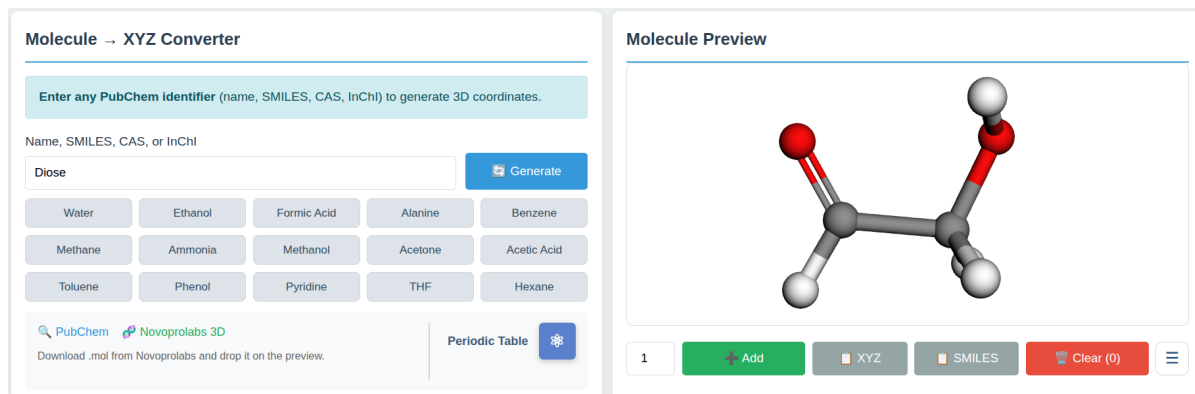


Figure 3: Generating initial coordinates for glycolaldehyde and water using the web interface.

A crucial step in the setup is superposition: both the glycolaldehyde and the water molecules are placed at the exact center of the simulation box, ensuring that the stochastic search starts from an unbiased configuration. The resulting setup should resemble Figure 4, so that the center of mass for each molecule occupies the same region of space before the simulation begins.

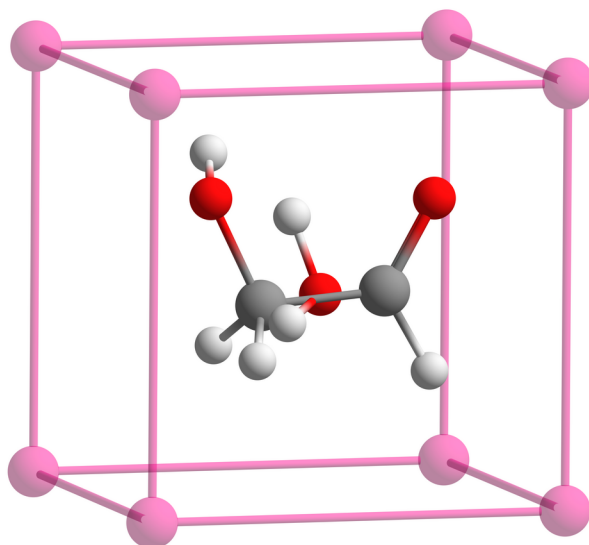


Figure 4: Initial simulation box containing glycolaldehyde and water superposed at the center.

In this example we select a preliminary run (Figure 5), which uses the GFN2-xTB method for both the single annealing stage and the subsequent candidate pre-optimization; motifs are then identified by the COSMIC clustering algorithm. This is an **optimization only** workflow (no frequency calculations), which keeps the process fast but does not guarantee that the resulting structures are true local minima.

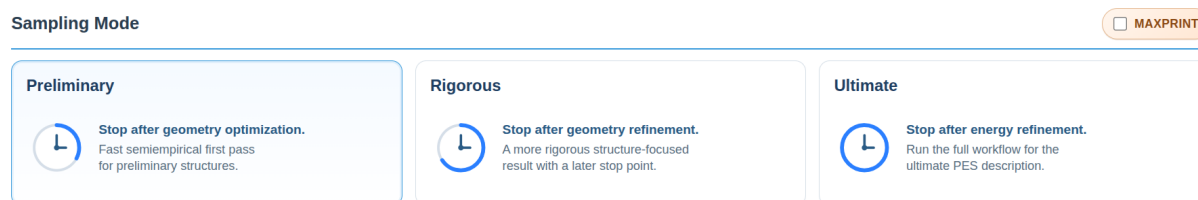


Figure 5: Preliminary sampling mode selection.

We enable the conformational option at the annealing theory level (Figure 6) and then download the `.asc` input file.

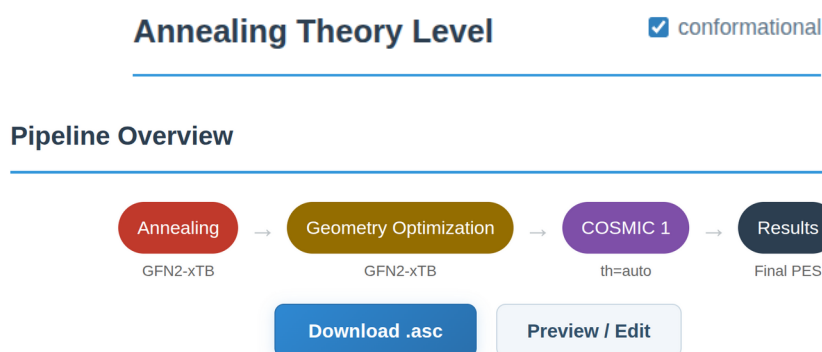


Figure 6: Conformational check and input download.

The necessary input file, `gaw.asc`, is provided in the examples folder.

```
/
└─ examples/
    └─ gaw.asc
```

To run the simulation, simply execute the program. This calculation is configured to use eight processor cores.

```
ascec gaw.asc
```

Upon execution, a progress bar is displayed.

```
=== COSMIC ASCEC ===
-----
Progress [.....] 0.0%
-----
[1/3] Annealing ...
```

Here, [1/3] indicates that the program is in the first of three main stages: Annealing, Geometry Optimization, and COSMIC screening.

Once the run is complete, the terminal will display a summary message.

```

=== COSMIC ASCEC ===
-----
Progress [#####] 100.0%
-----
[1/3] Annealing ✓
[2/3] geometry_optimization ✓
[3/3] cosmic (80→37) ✓

Workflow finished
Start: 2026-06-23 17:43:40
End: 2026-06-23 17:44:17
Total wall time: 36s 835ms
-----
Warning: No true minima can be assured without frequency calculations.
Protocol summary saved to protocol_summary.txt

✓ Miniprint: 23.8 MB (use --maxprint to keep all files)

```

In approximately 37 seconds, ASCEC autonomously executed a three-stage pipeline:

1. **Annealing** (66.3%): The stochastic search explored the configurational and conformational space of the glycolaldehyde–two-water complex using GFN2-xTB energies. Starting from a “Big Bang” superposition, it generated **80 accepted configurations** while cooling the system from 300 K down to near 0 K.
2. **Geometry Optimization** (33.7%): All 80 candidate structures were optimized at the GFN2-xTB level, relaxing each configuration into the nearest local energy basin and producing a set of pre-optimized structures.
3. **COSMIC** (< 0.1%): The clustering algorithm analyzed the 80 optimized structures using a reduced physicochemical feature vector and identified **37 motifs**, i.e., topologically distinct families of hydrated glycolaldehyde.

A representative subset of these motifs is shown in Figure 7; each of them corresponds to a different interaction pattern between water and glycolaldehyde, with a certain degree of redundancy remaining.

Note the warning in the terminal output. Because this workflow uses **Opt** (optimization only, without frequency calculations), the COSMIC module operates with a **reduced feature vector**. This is sufficient for a rapid preliminary survey of the PES, comparable to the level of analysis performed by many conformational-sampling programs. However, without frequency analysis the structures cannot be rigorously confirmed as true local minima, since some may correspond to saddle points. For publication-quality results, a complete **Opt+Freq** workflow should be used (as demonstrated in the Water Hexamer example); this yields a fully characterized PES with verified minima and a Boltzmann-weighted population distribution. The trade-off between speed and rigor is ultimately the user’s decision and depends on the goals of the study.

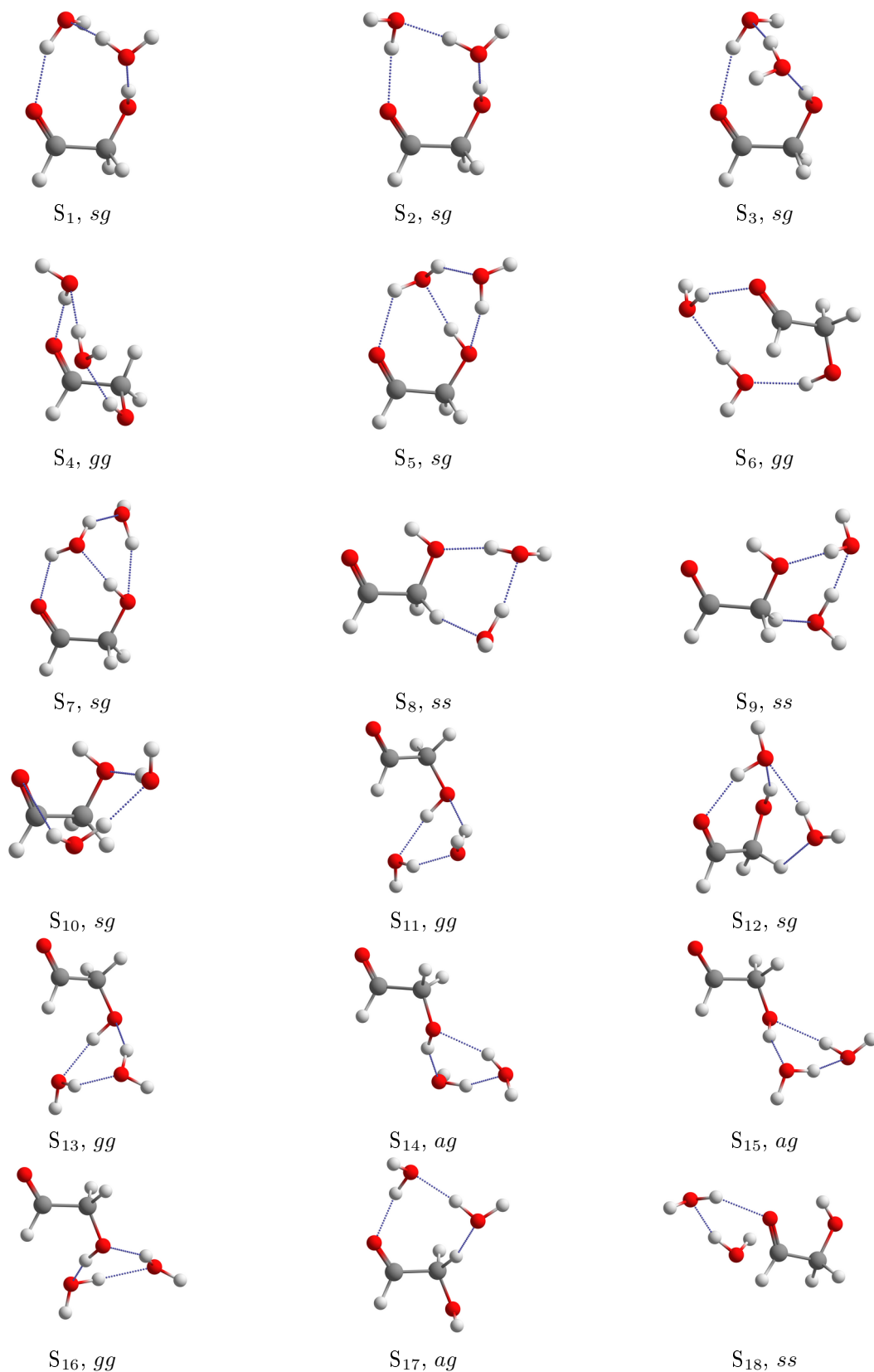


Figure 7: First 18 of the 37 preliminary glycolaldehyde-water motifs.

7 Water Hexamer

This section demonstrates the entire COSMIC-ASCEC workflow using the web generator to build the input file from scratch. Unlike the Quick Start example, this pipeline includes a geometry refinement stage, producing a fully characterized PES with verified minima and Boltzmann populations. We use the water hexamer (H_2O)₆ as a well studied benchmark system [10].

7.1 Molecular Coordinates Generation

Open the Web Input Generator at <https://manuel2gl.github.io/qft-cosmic-ascec/> (or run `ascec input` in the terminal). Using Smart Mode, search for "water" in PubChem to fetch its 3D structure, then add six copies to the simulation box. (Figure 8).

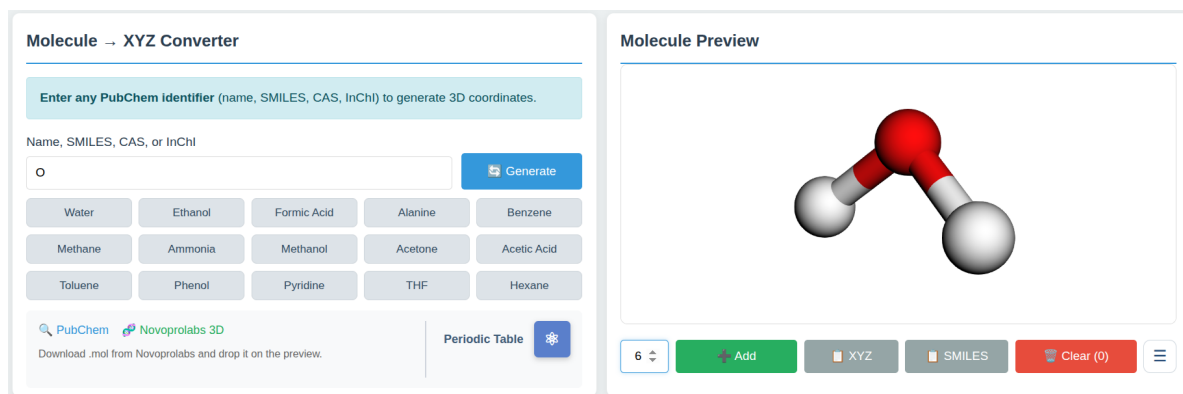


Figure 8: Generating six water molecules using the PubChem integration.

7.2 Mode Selection

The web generator provides several predefined modes. A **preliminary** run uses no replicas and a single low cost level of theory for a quick mapping of the PES. The **rigorous** and **ultimate** modes add a higher level refinement stage, producing a fully resolved PES with any level of theory available on the backend.

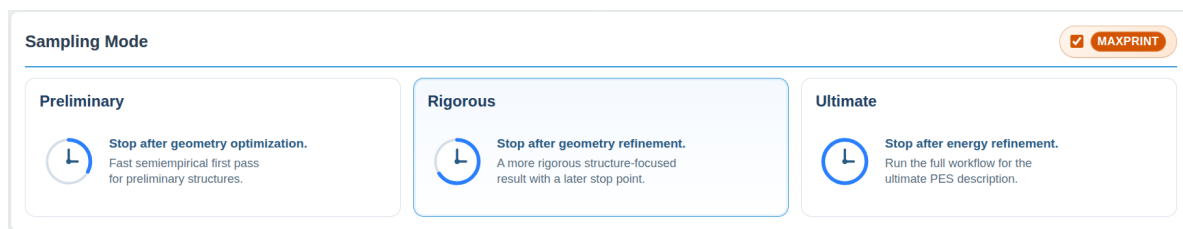


Figure 9: Rigorous mode selection.

As shown in Figure 9, the **maxprint** option is enabled by default in rigorous and ultimate modes: because these workflows involve expensive calculations, the user may prefer to retain all intermediate I/O files for further analysis. If it is disabled, **miniprint** is used instead and only the I/O files of the final ensemble are kept, saving disk space.

7.3 Simulation Box

The simulation box defines the cubic region where molecules are confined during the annealing search. The key parameter is the **effective packing percentage** ϕ , which controls how full the box is: a lower value produces a more dilute (spacious) box, while a higher value makes it denser. The default is 30% for preliminary runs, and 20% for rigorous and ultimate modes, which provides additional free space for the molecules to reorganize during the more exhaustive search. The algorithm that converts ϕ into a box length, together with the underlying geometric reasoning, is described in detail in Section 10.

The web generator (and ASCEC itself) automatically estimates the appropriate box size based on the molecular volumes and, for systems with hydrogen bond donors and acceptors, the additional space required by the H-bond network. For the water hexamer at the 20% effective packing used in rigorous mode, the suggested box length is approximately 7.3 Å (Figure 10). A more detailed box analysis using the standalone `ascec box` command is presented in Section 11.

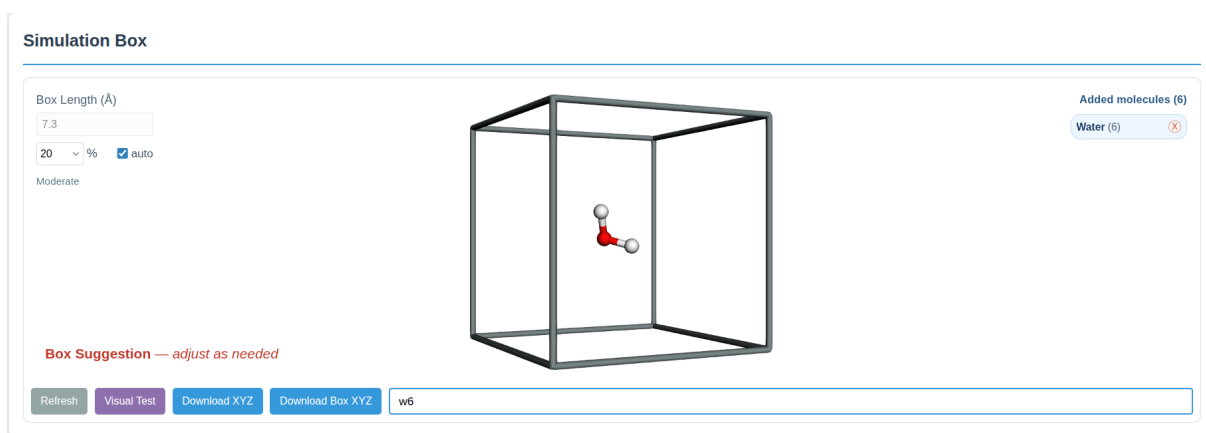


Figure 10: Simulation box setup for the water hexamer at 20% effective packing ($L \approx 7.3$ Å).

A visual test mimics the random rigid body moves performed by the ASCEC module, letting the user check whether the suggested box is adequate. This helps avoid both compressed boxes, which leave no room for cluster formation, and oversized boxes, in which the system cannot reliably evolve during the annealing procedure.

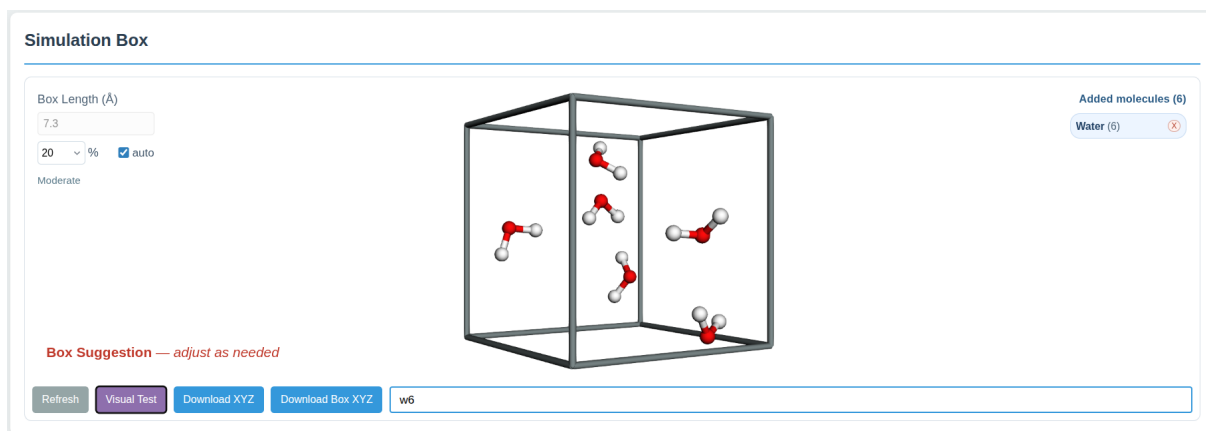


Figure 11: Water hexamer simulation box visual test

7.4 Annealing Parameters

Smart Mode provides a guided interface for selecting the annealing settings (Figure 12).

It supplies sensible defaults that the user can override as needed. The key parameters are:

- **Conformational:** Controls sampling of internal (torsional) degrees of freedom. For a *single* fragment it is enabled automatically with a sampling percentage of 100%. For *multiple* fragments it is optional (off by default); when enabled it defaults to 30%.
- **Annealing method:** GFN2-xTB semiempirical, for fast energy evaluations during the stochastic search.
- **nprocs:** Number of processors used for the QM calculations.
- **Concurrent & Replicas:** Three concurrent (triplicated) annealing runs to improve sampling coverage.
- **Charge & Multiplicity:** The system's overall charge and spin multiplicity; the GUI proposes a multiplicity, but it should always be reviewed.
- **Temperature schedule:** Geometric quenching, which gradually lowers the acceptance probability of high energy states.
- **Box:** With auto box enabled, the packing fraction can be adjusted; with it disabled, a custom box length can be entered.

Annealing Theory Level				Quenching & Box Settings			
☐ conformational				☒ auto box			
Theory	Method	Basis		Route	Initial T (K)	Factor (%)	Steps
Standalone ...	GFN2-xTB	(none)		Geom...	500	5	100
nprocs	Concurrent	Charge	Mult.	Replicas	Packing (%)	Box (Å)	
1	3	1	1	3	20	10.0	

Figure 12: Annealing parameter selection.

7.5 Geometry optimization and refinement

A geometry optimization is first carried out with the GFN2-xTB semiempirical method to relax the accepted structures and enable a first COSMIC screening.

The refinement step re-optimizes the unique motifs identified by the first clustering pass at a higher level of theory (Figure 13). For this run we use **wB97X-D3BJ/def2-TZVPP** with a full **Opt+Freq** treatment, which ensures that the final structures and their relative energies are accurate enough for meaningful thermodynamic analysis. Crucially, the frequency calculation confirms each structure as a true local minimum.

A second COSMIC analysis is then applied to the refined ensemble to produce the final set of unique motifs and the Boltzmann population distribution.

Geometry Optimization

Theory	Method	Basis
Standalone ...	GFN2-xTB	(none)

nprocs	Concurrent	MaxIter	Job
1	8	200	Opt

Geometry Refinement optional

Theory	Method	Basis
DFT	wB97X-D3BJ	def2-TZVPP

nprocs	Concurrent	MaxIter	Job
12	2	120	Opt Freq

Figure 13: Geometry optimization & Refinement theory level.

7.6 Energy Refinement & COSMIC Screening

An optional energy refinement is reserved for the ultimate mode, which is why it is disabled in this rigorous run. The COSMIC screening parameters are also set here: in **automatic** mode the protocol determines the best clustering threshold during the first COSMIC pass (after the geometry-optimization stage; see Section 11.3.4 for details) and reuses that same threshold for the final COSMIC stage (Figure 14).

Energy Refinement optional

Theory	Method	Basis
Post-HF	DLPNO-CC...	aug-cc-pVTZ

nprocs	MaxCore (MB)	Job
1	50000	SP (Single ...)

COSMIC Screening same for opt/ref

Opt th	Opt RMSD	Opt -j
automatic	off	4

Ref th	Ref RMSD	Ref -j
opt	off	4

Figure 14: Energy Refinement & COSMIC Screening.

7.7 Pipeline Review and Execution

Before generating the input file, the interface shows the complete execution pipeline (Figure 15). This summary allows you to verify each stage before downloading the **.asc** file.

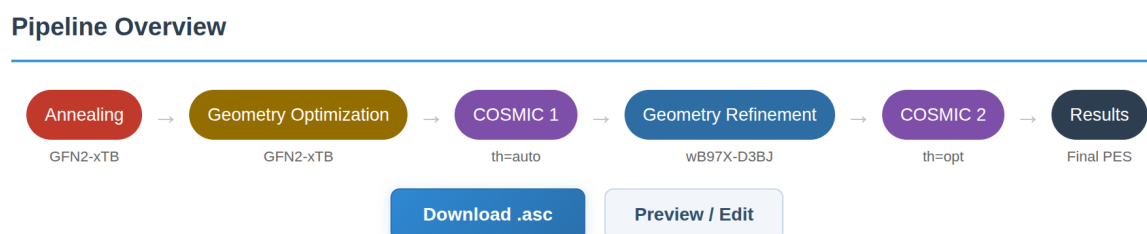


Figure 15: Complete pipeline review showing all workflow stages.

Download the generated `.asc` file and place it in your working directory, adjust the orca input lines as needed.

```
#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*
```

Then execute the protocol:

```
ascec w6.asc
```

Because the full pipeline can run for hours or days, it is normally left running unattended. There are two ways to detach it from the terminal: launch it in the background with the shell ampersand, (`ascec w6.asc &`), or simply press **Ctrl+D** while the run is attached.

On **Ctrl+D** the program relaunches itself fully detached from the terminal session and prints the background PID; the job keeps running and the shell is freed. Closing the terminal does not stop a detached job.

The `ascec status` command opens an interactive viewer of all ASCEC jobs on the machine (a registry shared across terminals), so a detached run can be inspected at any time:

```
ascec status
```

It lists the running and recent jobs with their ID, PID, input file, and start time. From the prompt you can attach to a job and watch its live progress (**V** <id>), terminate it (**K** <id>), refresh the list (**R**), or quit the viewer (**Q**), quitting the viewer leaves the jobs running.

```
ASCEC STATUS (2026-06-25 20:28:46)
```

```
Running:
```

ID	PID	INPUT FILE	STARTED
1	1131777	w6.asc	2026-06-19 16:54:48

```
[V <id>] Attach/View  [K <id>] Kill    [R] Refresh  [Q] Quit
```

```
Choice: v1
```

```

=== COSMIC ASCEC ===
-----
Progress [#####.....] 68.6%
-----
[1/5] annealing ✓
[2/5] geometry_optimization ✓
[3/5] cosmic ✓ (313→49)
[4/5] geometry_refinement 21/49 ...

Attached: job 1 (at.asc)   updated: 2026-06-25 14:29:05
Ctrl+C or Ctrl+D to detach (job keeps running)
Input realpath: /home/protio/Documentos/at/at.asc

```

The automated protocol will sequentially run all stages: Annealing, Geometry Optimization, COSMIC, Geometry Refinement, and Final COSMIC. Depending on the theory levels and number of replicas, the computation may take several hours to a few days.

The run is resumable. Each stage checkpoints its progress to a cache file, so if the workflow is interrupted (or deliberately stopped), simply re-issuing the same command, `ascec w6.asc`, picks up where it left off: completed stages are read from the cache and skipped, and execution continues from the first unfinished stage.

If you have another protocol to run, you can queue it behind a job that is already running instead of waiting for it to finish. The `after` keyword takes the PID of the running job (shown in `ascec status`) and holds the new run until that job completes.

```
ascec input2.asc after 1131777
```

```

ASCEC queued in background (PID 3964514) - holding until PID 1131777 finishes,
then it starts automatically.
Track or cancel:  ascec status

```

7.8 Results

The automated protocol funnels the raw stochastic search down to a compact set of fully characterized minima. The three triplicated annealing runs produced **215 accepted configurations**, all of which were pre-optimized at the GFN2-xTB level. A first COSMIC pass reduced this ensemble to **39 motifs**, which were then refined at the wB97X-D3BJ/def2-TZVPP level with frequency analysis. A second COSMIC pass on the refined ensemble yielded **22 unique minima**. The complete pipeline ran in roughly **1 day and 3 hours** on a standard workstation, with the DFT refinement stage accounting for $\sim 99.8\%$ of the wall time; the annealing and pre-optimization stages together took only a few minutes.

The final clustering applied a topological threshold of $\tau = 2.0$. Its built-in validation initially flagged 23.1% of the refined structures as critical (incomplete or non-minimum); these were automatically re-processed, and after the redo passes the critical fraction dropped to 0%, confirming that all 22 representatives correspond to genuine local minima. Figure 16 shows the threshold diagnostic that selects τ and the resulting dendrogram of the 22 clusters.

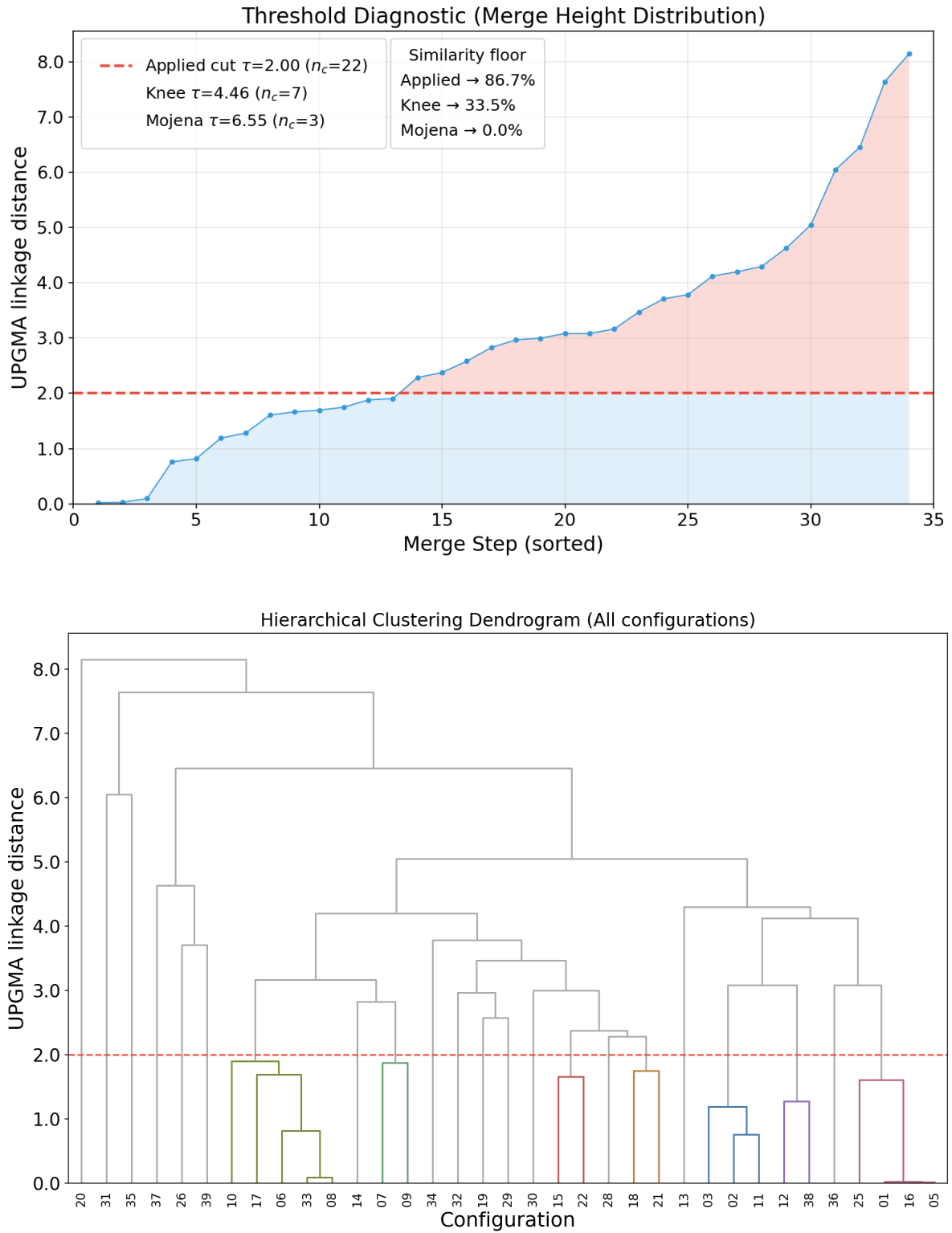


Figure 16: Final COSMIC analysis of the refined ensemble: the threshold diagnostic used to fix $\tau = 2.0$ and the dendrogram grouping the refined structures into 22 unique motifs.

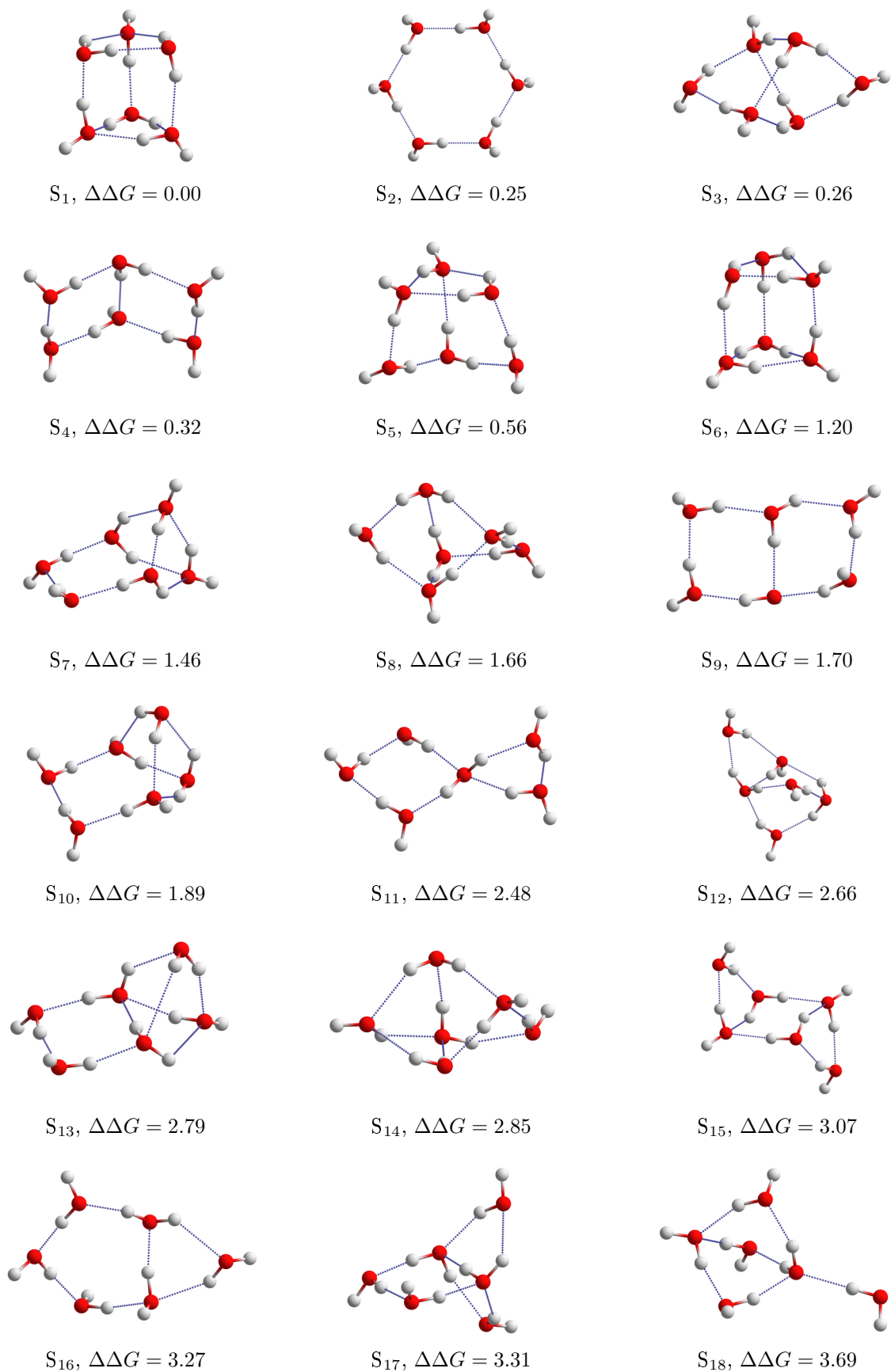


Figure 17: First 18 of the 22 refined water hexamer minima, labeled with their relative Gibbs free energy $\Delta\Delta G$ (kcal/mol) at 298.15 K.

The Boltzmann population (298.15 K) is dominated by a few low lying minima: the five most stable motifs hold $\sim 88\%$ of the population, all within ~ 0.6 kcal/mol of the global minimum. A subset is shown in Figure 17.

The final ensemble is only as reliable as the refinement method: faster methods (semiempirical, GFN2-xTB) give a cheap preliminary survey, but accuracy depends on the high-level stage. Against reported DFT and MP2 data for small water clusters [10], the ASCEC protocol recovers the known dominant motifs at a reasonable cost on a standard workstation.

Computational efficiency. The hierarchical design reserves expensive quantum-chemical work for the few structures that matter (Figure 18). The ASCEC search and xTB pre-optimization explore the space and generate hundreds of candidates for a small fraction of the runtime, the water hexamer’s preliminary survey finishes in ~ 3 minutes, while rigorous DFT geometry optimization and, in the ultimate protocol, DLPNO-CCSD(T) energy refinement dominate ($\gtrsim 99\%$) the wall time.

Since COSMIC collapses the ensemble to a handful of motifs *before* these steps run, costly calculations are minimized while the dominant minima are retained, letting the user trade accuracy against cost (preliminary, rigorous, ultimate) predictably.

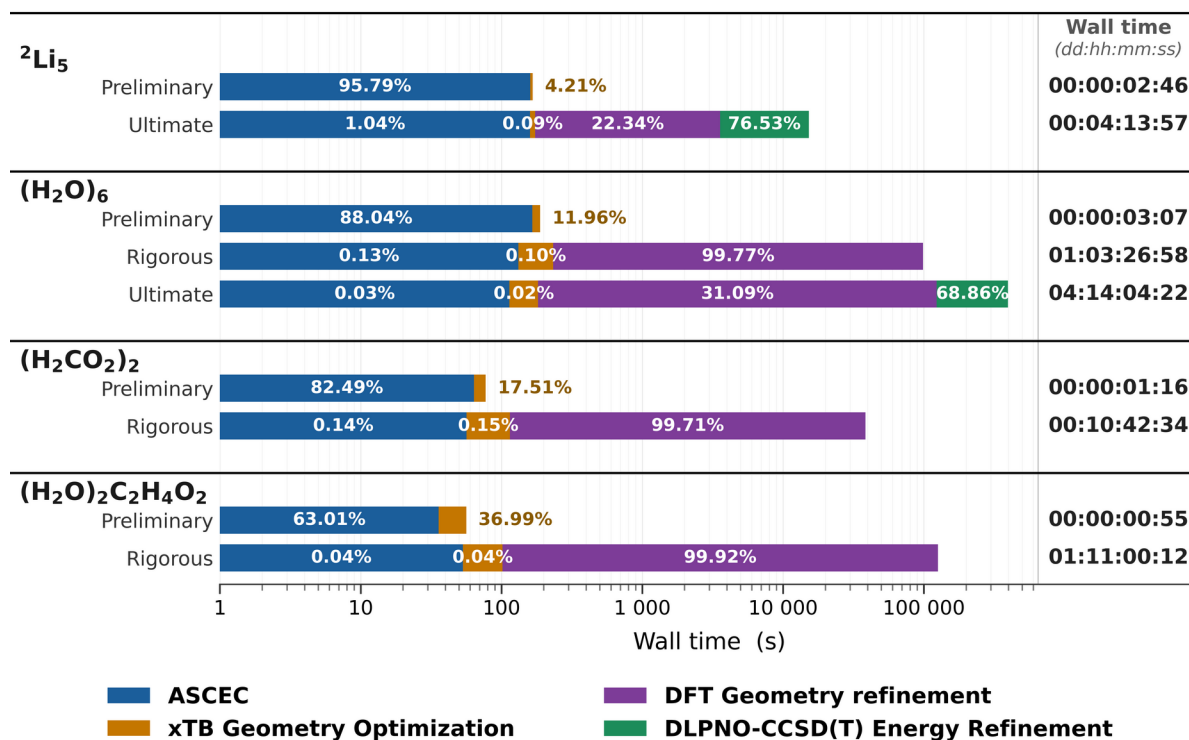


Figure 18: Wall time breakdown of the COSMIC-ASCEC pipeline stages for four benchmark systems, including the water hexamer $(\text{H}_2\text{O})_6$. Total wall time spent in each stage shown in logarithmic time axis.

8 Advanced Usage & Reference

9 Input File Format

The ASCEC input file (typically saved with an `.asc` extension) is a comprehensive "master" document. To make it easy to read and edit, it is logically divided into three main blocks: the **Simulation Parameters**, the **Execution Protocol**, and the **Embedded QM Templates**.

Note: The easiest way to generate this file without worrying about formatting is by using the [ASCEC Web Input Generator](#).

9.1 Part 1: Simulation Parameters & Coordinates

The first block of the input file relies on strict positional formatting. It defines the physical conditions of the annealing simulation and the initial coordinates of the molecules.

```
# Input for water pentamer Simulation

1 10          # Line 1: Simulation Mode & Number of Config
              # 1: Annealing; 0: Random configurations
7.0          # Line 2: Simulation Cube Length (Angstroms)
2            # Line 3: Annealing Quenching route
              # (1: Linear, 2: Geometrical)
100.0 10.0 50 # Line 4: Linear Quenching Parameters (Ti, dT, steps)
500.0  5.0 100 # Line 5: Geometric Quenching Parameters (Ti, %, steps)

3000 10      # Line 6: Max MC Cycles per T and floor value
1.0 1.0      # Line 7: Max Displacement (A) & Max Rotation (radians)
0 120        # Line 8: Conformational sampling (%) & Max dihedral (deg)

3 xtb        # Line 9: QM Program Index & alias
              # (1: Gaussian, 2: ORCA, 3: xTB)
GFN2-xTB     # Line 10: Hamiltonian & Basis Set
1 4          # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1          # Line 12: Charge & Spin Multiplicity

5            # Line 13: Number of Molecules (nmo)

# Lines below: Molecule Definition
# (Num Atoms, Label, then xyz coordinates, separated by *)

*
3
Water1
O   -0.294733   -0.218200    0.154167
H   -0.017333    0.674700    0.408567
H    0.312067   -0.456500   -0.562733
*
3
Water2
O   -0.294733   -0.218200    0.154167
H   -0.017333    0.674700    0.408567
H    0.312067   -0.456500   -0.562733
*   ... (Water3-5)
```

Table 2: Detailed description of ASCEC input parameters (Lines 1-13).

Line	Parameters	Description
1	[mode] [nconfigs]	1 : Annealing, 0 : Random Generation. nconfigs is only used if mode is 0.
2	box_length	Length of the cubic simulation box edge in Å.
3	route	Temperature schedule: 1 (Linear) or 2 (Geometric).
4	[Ti][dT] [steps]	Linear: Start Temp (K), Step size (K), Number of steps.
5	[Ti] [fac] [steps]	Geometric: Start Temp (K), Reduction factor (%), Steps.
6	[max][floor]	Maximum Monte Carlo cycles per temperature step and floor value as it decreases (0.1) with each step.
7	[disp] [rot]	Max translation distance (Å) and max rotation (radians). Maximum displacement for each mass center, and maximum rotation of the principal axes of each molecule. The second field must be 0 for atomic clusters.
8	[prob][dih]	Probability of conformational move (%) and max dihedral change (°).
9	[code] [alias]	QM Program ID (1 =Gaussian, 2 =ORCA, 3 =xTB) and the command alias to run it.
10	[method][basis]	Hamiltonian/Method (e.g., gfn2 , b3lyp) and basis set (e.g., def2-SVP). For semiempirical methods the basis field is ignored; consult the ORCA manual for the correct syntax.
11	[cpu_Q] [cpu_A]	Processors passed to the QM program, and (optional) processors for ASCEC logic and parsing.
12	[chg][mult]	Total system Charge and Spin Multiplicity.
13	nmol	Number of molecules defined. For atomic clusters each atom must be considered a molecule. If atoms interacting with molecules are considered, then each atom not being part of a molecule must be considered a molecule.

9.2 Part 2: The Automated Protocol & Embedded Templates

Following the molecular coordinates, the file contains the **#Protocol** block and the **embedded QM templates**. The protocol is a sequence of comma separated steps. The example below is a complete ultimate protocol; stopping after the first **cosmic** gives a preliminary run, and after the second a rigorous one.

```
# Protocol
# (comma = continue, dot = pause; --flags apply to the whole run)

.asc,
--maxprint,
r3 --concurrent=3,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4,
eref input3,
cosmic --th=opt -j4
```

Workflow: Annealing → Opt → COSMIC → Ref → COSMIC → Eref → COSMIC

Each optimization or refinement step names a template (`input1`, `input2`, `input3`) that is **embedded at the bottom** of the same file, using the header and placeholder syntax of Section 11.2.1. The three tiers here are GFN2-xTB (`#xtb`), wB97X-D3BJ/def2-TZVPP with frequencies (`#orca`), and a DLPNO-CCSD(T)/aug-cc-pVTZ single point (`#orca`):

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*

#orca input3
#name
! DLPNO-CCSD(T) aug-cc-pVTZ def2/J AutoAux RIJCOSX TightPNO TightSCF
%scf
  MaxIter 250
end
%pal
  nprocs 1
end
%maxcore 50000
* xyz 0 1
#
*
```

Understanding the Execution Pipeline

Upon parsing the `.asc` file and its embedded templates, ASCEC executes the stages in order. It begins with triplicated **Annealing** (`r3 -concurrent=3`) to generate raw candidates, then **pre-optimizes** the whole pool with the GFN2-xTB `input1` template (`opt`) and applies the first **COSMIC** screening.

The surviving motifs are **refined** at wB97X-D3BJ/def2-TZVPP with frequencies (`ref input2`) and screened again — the `-th=opt` on this second `cosmic` re-uses the threshold resolved by the first. Finally an **energy refinement** (`eref input3`) computes DLPNO-CCSD(T) single points on the DFT geometries, and a last **COSMIC** pass delivers the unique minima and their Boltzmann populations. Ending the protocol after the first or second `cosmic` yields the preliminary or rigorous description instead.

***Note:** From this point forward, the advanced sections of this manual explain each of these stages in detail — box sizing, the feature vector and thresholds, the templates and launchers, and how to run each function on its own.*

10 Box Size Suggestion

ASCEC performs simulated annealing inside a hard-walled cubic box of side length L . Molecular (or atomic) centres of mass are confined within the walls, and trial moves that place a centre outside the box are rejected. Choosing L correctly is critical: a box that is **too small** compresses the molecules, prevents free rotation, and introduces artificial steric clashes that distort the energy landscape; a box that is **too large** wastes annealing time sampling empty space and converges slowly.

The whole procedure is governed by a single, intuitive parameter: the **packing fraction** ϕ (given as a percentage, e.g. `-box20` for $\phi = 20\%$), which answers the question “*how full do you want the box to be?*” Two methods compute an **effective volume** V_{eff} , but both feed the same box-length formula:

$$L = \left(\frac{V_{\text{eff}}}{\phi} \right)^{1/3} \quad (6)$$

Method	When used	V_{eff}	What it captures
A	System has primary H-bonds	$V_{\text{mol}} + V_{\text{HB}}$	Hull volume + H-bond ghost cylinders
B	No primary H-bonds	L_{diag}^3	Diagonal-derived volume from molecular extents

Table 3: The two routes to the effective volume. The selection is automatic.

Method A is preferred whenever the system can form hydrogen bonds, because it captures the directional, rigid geometry of H-bonded networks. For systems with no primary H-bonds (van der Waals clusters, noble-gas clusters, metal clusters) Method B estimates the required volume from the molecular extents. Both are fully general: they work for any number of molecules and for atomic, molecular, or mixed systems, requiring only the 3D coordinates and atomic numbers.

10.1 Molecular Volume via Convex Hull

Each atom is treated as a hard sphere whose radius is its **single-bond covalent radius** (Cordero *et al.*, 2008). The spheres of one molecule are assembled in their correct 3D geometry, and the **convex hull** of that surface — the smallest convex polyhedron enclosing it, as if a sheet were stretched tightly around the cluster — defines the molecular volume V_{mol} . No shape assumption is made: a single atom yields a sphere, a planar molecule a flattened polyhedron, a globular cluster a roughly spherical one.

Covalent radii (not van der Waals radii) are used because they define the irreducible hard core that other molecules cannot overlap. The softer van der Waals contact region is part of the *free space* between molecules, which is precisely what the packing fraction ϕ controls; using van der Waals radii would double-count that space and inflate the box.

Element	H	C	N	O	F	S	Cl	Ar	Au
$r_{\text{cov}} / \text{\AA}$	0.31	0.73	0.71	0.66	0.57	1.05	1.02	1.06	1.36

Table 4: Representative single-bond covalent radii used as sphere radii.

Surface points: the Fibonacci sphere

For a molecule with N atoms at positions $\mathbf{r}_k$ and covalent radii R_k , a set of points approximating the outer envelope of each atomic sphere is generated with the **Fibonacci sphere** algorithm, which places $N_s = 50$ nearly uniformly distributed points per atom. For $i = 0, 1, \dots, N_s - 1$:

$$\theta_i = \arccos\left(1 - \frac{2(i + 0.5)}{N_s}\right), \quad (7)$$

$$\varphi_i = \frac{2\pi i}{\Phi}, \quad \Phi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \text{ (golden ratio)}, \quad (8)$$

$$\hat{\mathbf{d}}_i = (\sin \theta_i \cos \varphi_i, \sin \theta_i \sin \varphi_i, \cos \theta_i), \quad (9)$$

and the surface point for atom k in direction i is $\mathbf{p}_{k,i} = \mathbf{r}_k + R_k \hat{\mathbf{d}}_i$, giving $50N$ points in total. Unlike a simple $\{-1, 0, +1\}^3$ lattice of directions, the Fibonacci distribution contains **no coplanar clusters** of points. This matters because two different convex-hull libraries (e.g. Qhull in Python versus the incremental hull used by the web generator) can triangulate coplanar regions differently and report slightly different volumes; the Fibonacci construction removes that ambiguity and is deterministic to IEEE 754 double precision, so both implementations agree.

Hull volume

The hull volume is obtained from the **signed-tetrahedron formula**. For a hull with n_F triangular faces, each with vertices $\mathbf{v}_1^{(f)}, \mathbf{v}_2^{(f)}, \mathbf{v}_3^{(f)}$:

$$V_{\text{mol}} = \left| \frac{1}{6} \sum_{f=1}^{n_F} \mathbf{v}_1^{(f)} \cdot (\mathbf{v}_2^{(f)} \times \mathbf{v}_3^{(f)}) \right|. \quad (10)$$

This is exact for any closed polyhedron; the only approximation is the discretisation of each atomic sphere into 50 points. For a single argon atom ($R = 1.06 \text{ \AA}$) the hull of 50 Fibonacci points gives $V_{\text{mol}} \approx 4.82 \text{ \AA}^3$, within 3.5% of the analytical sphere volume $4.99 \text{ \AA}^3$ — a slight, conservative underestimate that is absorbed by the free-space fraction $(1 - \phi)$. If a hull library is unavailable, a bounding-box fallback $V_{\text{mol}} \approx 0.52 V_{\text{bbox}}$ is used, the factor 0.52 approximating the typical hull-to-box ratio for organic molecules.

10.2 Molecular Extent

The **molecular extent** E is the largest distance spanned by the molecule, including the covalent radii of the two terminal atoms:

$$E = \max_{j < k} (\|\mathbf{r}_j - \mathbf{r}_k\| + R_j + R_k). \quad (11)$$

It is the effective “diameter” of the molecule — the largest dimension it can occupy regardless of orientation. For a single atom, $E = 2R$.

10.3 Hydrogen-Bond Network Volume (Method A)

Hydrogen bonds are directional and rigid, operating at ≈ 2.5 Å and forcing molecules into open arrangements with large voids (the same effect that makes ice less dense than liquid water). The hull volume measures only the molecular skeleton, so Method A adds a cylindrical “ghost volume” per potential H-bond to signal that the system needs extra room.

A system is treated as hydrogen-bonded when, across all molecules, there is at least one **donor** and one **acceptor**. Every H atom counts as a donor, and every N, O, F, P, S, Se, Cl, Br or I atom counts as an acceptor. For each molecule the number of potential bonds is $n_{\text{HB}} = \min(n_{\text{donors}}, n_{\text{acceptors}})$, and each is modelled as a cylinder of length $\ell_{\text{HB}} = 2.5$ Å and radius $r_{\text{HB}} = 1.2$ Å:

$$V_{\text{HB}}^{(\text{bond})} = \pi r_{\text{HB}}^2 \ell_{\text{HB}} = \pi (1.2)^2 (2.5) \approx 11.31 \text{ Å}^3, \quad V_{\text{HB}}^{(\text{mol})} = n_{\text{HB}} V_{\text{HB}}^{(\text{bond})}. \quad (12)$$

10.4 Effective Volume and Box Length

Summing over all M molecules (copies included) gives the totals $V_{\text{mol}}^{\text{total}} = \sum_i V_{\text{mol},i}$, $V_{\text{HB}}^{\text{total}} = \sum_i V_{\text{HB},i}^{(\text{mol})}$, and $\sum_i E_i$. The effective volume is then selected automatically:

$$\text{Method A (H-bonded): } V_{\text{eff}}^{(A)} = V_{\text{mol}}^{\text{total}} + V_{\text{HB}}^{\text{total}}, \quad (13)$$

$$\text{Method B (no H-bonds): } V_{\text{eff}}^{(B)} = L_{\text{diag}}^3, \quad L_{\text{diag}} = \frac{\sum_i E_i}{\sqrt{3}}. \quad (14)$$

In Method B the numerator $\sum_i E_i$ is the total span of all molecular extents laid end to end; treating that sum as the body diagonal of a cube and dividing by $\sqrt{3}$ converts it to a cube edge, giving a “natural geometric size” for the system. The box length then follows from Eq. 6; for Method B it simplifies to $L^{(B)} = L_{\text{diag}}/\phi^{1/3}$, so that $\phi = 1$ recovers $L = L_{\text{diag}}$ and smaller ϕ grows the box to add free space.

Why the cubic root matters

Because $L = (V_{\text{eff}}/\phi)^{1/3}$, doubling the number of molecules doubles V_{eff} but increases L by only $2^{1/3} \approx 1.26$, which is correct since the box volume scales as L^3 . A naive linear rule $L \propto \sum_i E_i$ would instead double L and produce a box $2^3 = 8$ times too large, an error that compounds rapidly with system size.

10.5 Choosing the Packing Fraction

The packing fraction answers “*what fraction of the box is effectively occupied?*” — molecular hulls plus H-bond cylinders in Method A, or the diagonal-derived reference volume in Method B. The same `-box20` therefore means the same thing for water, methane, argon, or a mixed system; the algorithm computes V_{eff} and picks the method without user intervention. The web generator’s preset modes set ϕ for you (30% for preliminary, 20% for rigorous and ultimate); when driving the command line directly, the following ranges are useful guidance:

ϕ	Density regime	Typical application
5%–10%	Very sparse	Initial conformational search
10%–20%	Sparse	Most molecular cluster studies
20%–30%	Moderate	Network or aggregate formation
30%–50%	Dense	Condensed-phase packing, crystal seeds
50%–65%	Very dense	Liquid-like; approaching the sphere-packing limit

Table 5: Recommended packing fractions by density regime.

11 Understanding Each Stage

Almost every user drives COSMIC-ASCEC through the automated pipeline (Workflow Mode, Section 13.2), where a single command chains annealing, optimization, clustering and refinement. This section explains *what each stage does* and gives the standalone command that runs it on its own — useful when you want to execute a single stage, inspect its intermediate output, or re-cluster an existing result set with different settings. Every stage uses the same files as the protocol: the `.asc` input, the embedded or external QM templates, and a launcher script (see the `examples/` folder).

11.1 Annealing and the Box

Before launching, it is worth checking the box. The `box` subcommand prints the effective-volume analysis of Section 10 and a table of recommended edge lengths, without running any QM:

```
ascec w6.asc box
```

```
(base) :~/.../examples/water_hexamer/orca6$ ascec w6.asc box
```

```
=====
Box length analysis
=====
```

```
1. Molecular volume and hydrogen bonding analysis:
```

```
-----
Number of molecules to place: 6
Total molecular volume: 6.46 Å³
Total H-bond network volume: 67.86 Å³
Total effective volume: 74.31 Å³
```

```
3. Current box analysis:
```

```
-----
Cube's length = 5.00 Å
Current effective packing: 59.5%
+ Molecular: 5.2%, H-bond network: 54.3%
Current free volume: 51 Å³ (40.5%)
Largest molecular extent: 1.13 Å
Extremely dense - may prevent proper H-bond formation
```

```
4. Recommendations for H-bonded systems:
```

```
-----
• For isolated clusters: 11.4 Å (5% effective packing)
```

- For cluster formation: 9.1 Å (10% effective packing)
- For network studies: 7.9 Å (15% effective packing)
- Includes space for H-bond network (avg. bond length: 2.5 Å)

The default 5.0 Å box is far too dense for this system, so we let ASCEC rewrite the length to a target packing fraction and, at the same time, replicate the run. The command below creates three independent replicas sized for 10% packing, together with a launcher that runs them in series:

```
ascec w6.asc r3 --box10
```

Each replica starts from the “**Big Bang**” configuration: all centres of mass superimposed at the box centre — the worst possible, and therefore least biased, starting point. ASCEC makes up to 100 random moves to find a first configuration whose energy is computable without atomic clashes, then anneals from there, writing the accepted structures, the energy trajectory (`tvse`), and the lowest-energy structure to each replica folder for later inspection. If all 100 attempts fail, the run stops with diagnostics instead of a trajectory.

11.2 Geometry Optimization

The annealing pool is large and highly redundant, so before any expensive refinement every accepted structure is relaxed with the fast semiempirical GFN2-xTB method [7]. This pre-optimization must preserve the topology of the PES, letting each structure fall into its nearest local minimum without merging distinct basins or discarding valid low energy isomers.

```
/
├─ annealing/
├─ geometry_optimization/
└─ w6.asc
```

11.2.1 Input Templates

A template is an ordinary QM input file for the target program, with a few `#` placeholders that ASCEC fills in for every structure. Each template, whether embedded at the bottom of the `.asc` file or supplied as a standalone `.inp`, begins with a header line that names the program *and* gives a label the protocol refers to:

```
#orca <label>      #xtb <label>      #gaussian <label>
```

A protocol step such as `opt input1` or `ref input2` then selects the block whose label matches; the header program also fixes the file extension written for that stage (`.inp`, `.xtb` or `.com`). The placeholders ASCEC recognises are shown in Table 1

Placeholder	Meaning
#orca xtb gaussian <label> #name	Block header: declares the QM program and the label to call (Optional) replaced by the structure's comment/filename, so each generated input is traceable.
#	A bare hash marks where the Cartesian coordinates are inserted.
#rescue(method[/freq][,num])	Method used to re-optimize failed structures in the redo loop (e.g. <code>#rescue(HF-3c/freq)</code> , <code>#rescue(GFN2-xTB/freq)</code>).
###	(Launcher only) splits the environment setup from the per job execution commands ASCEC appends.

Table 6: Template and launcher placeholders substituted by ASCEC.

I. ORCA template. Written in native ORCA syntax. The bare # sits between `* xyz <chg>` and the closing `*`, exactly where the coordinate block is injected:

```
#orca input1
#name
#rescue(Native-GFN2-xTB/num)
! Opt GFN2-xTB TightSCF NumFreq PAL8
%geom
  maxiter 5000
end
* xyz 0 1
#
*
```

II. xTB template. For standalone xTB the run directives are given as #-prefixed lines (method, processors, iterations); ASCEC supplies the geometry automatically, so no `* xyz *` block is needed:

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200
```

11.2.2 Running and Sorting

The `opt` command scans the `annealing/` folder for result `.xyz` files, injects each configuration into the template at the `#` placeholder, and writes one QM input per structure plus a launcher that runs them in series:

```
ascec opt gfn2_orca.inp launcher_orca.sh
```

Replicas can be processed separately or combined into a single numbered pool. Running the generated `launcher_orca.sh` performs the calculations. When they finish, `sort` organizes the many output files into subfolders, collects the `.out` files for clustering, and writes a wall time summary (skip it with `--nosum`):

```
ascec sort
```

11.3 COSMIC Clustering

11.3.1 Configuration Screening

The `sort` command (executed previously) extracts all valid output files (e.g., `.out` or `.log`) into a dedicated subdirectory. This prepares the dataset for the topological analysis.

```

/
├── annealing/
├── geometry_optimization/
├── cosmic/
│   ├── orca_out_416/
│   │   ├── opt_conf_1.out
│   │   ├── ..., .4 opt_conf_416.out
│   └── w6.asc

```

To initiate the topological analysis, navigate to the `cosmic` directory. The script is executed using the `cosmic` command, which accepts two primary control arguments:

- **--th (Threshold):** Defines the clustering granularity. This value represents the maximum Euclidean distance in the physicochemical feature space for two structures to be considered part of the same motif.
- **-j (Jobs):** Specifies the number of CPU cores to use for parallel parsing of the output files. If not specified, the script will attempt to use all available cores.

Understanding the Threshold:

A lower threshold creates tighter, more homogeneous clusters (distinguishing subtle geometric variations), while a higher threshold yields broader, more inclusive families. The choice of threshold depends on the system size and the desired resolution:

Granularity	Value (τ)	Typical Use Case
Strict / Tight	0.5 - 1.5	Small/Rigid Systems. Distinguishes between specific rotamers or iso-energetic conformers with slight dihedral changes.
Standard	2.0 - 2.5	General Purpose. Default for molecular clusters (e.g., water hexamers). Balances distinct topology against vibrational noise.
Loose / Broad	> 3.0	Large/Flexible Systems. Groups broad structural families (e.g., folding patterns) while ignoring local side-chain fluctuations.

Table 7: Recommended clustering threshold (τ) values.

11.3.2 Feature Vector Technicalities

Three implementation details govern how the raw physicochemical descriptors of Eq. 4 become the standardized vectors that are actually clustered.

Hydrogen-bond descriptors. The components n_{HB} , $\overline{d_{\text{HB}}}$, $\overline{\theta_{\text{HB}}}$ and $\sigma_{d_{\text{HB}}}$ are computed from a strictly geometric hydrogen-bond test applied to each structure. For every H covalently bound to a donor D (N, O or F), an acceptor A (also N, O or F) is counted as hydrogen-bonded when the $\text{H}\cdots\text{A}$ distance lies in $[1.4, 3.2]$ Å and the $\text{D}-\text{H}\cdots\text{A}$ angle is at least 30° . These are the same criteria used throughout the cluster literature. Note that this detection is deliberately *stricter* than the broad, count-based donor/acceptor heuristic used only to *size the box* (Section 10.3): the box heuristic over-counts on purpose to leave room, whereas the feature vector records the H-bonds a structure actually forms.

Small standard deviations. Z-standardization divides each descriptor by its standard deviation across the pool. If a descriptor is essentially constant, that division would inflate tiny numerical noise into enormous z-scores and dominate the Euclidean distance. COSMIC guards against this by *dropping* any feature column whose population standard deviation falls below a minimum threshold, or whose total spread across the pool is smaller than a user-set absolute tolerance. A dead column carries no discriminating information, so removing it entirely (rather than retaining it as noise) keeps the distances finite and meaningful; the dropped features are listed in the verbose log.

Pearson similarity floor. For two weighted, z-standardized vectors the Euclidean distance and the Pearson correlation r between their profiles are linked by $d^2 = 2N_f(1 - r)$, where N_f is the effective number of active features (the sum of the column variances after weighting). COSMIC inverts this, $r = 1 - d^2/(2N_f)$, to report, for every cluster, the lowest percentage similarity between any member and its representative — the **similarity floor**. A high floor (e.g. $> 90\%$) certifies that a family is internally homogeneous, giving a direct, quantitative check on the clustering quality.

Partial weights (preliminary runs). By default every feature carries equal weight ($w^k = 1$), which keeps COSMIC method-agnostic. The flag `--partialweights` instead applies a tuned, reliability-based profile that down-weights the descriptors most sensitive to method noise and atom indexing — dipole moment (0.6), the hydrogen-bond descriptors (0.7), the HOMO energy and highest vibrational frequency (0.85) — while leaving the deterministic geometric quantities (electronic energy, nuclear repulsion, rotational constants) at 1.0, so that geometrically identical structures do not look artificially distinct at a noisy level of theory. The web input generator adds `--partialweights` automatically to the COSMIC step of *preliminary* runs (where the whole pipeline stays at the semiempirical/xTB level); it is a deliberate choice for that specific case, not a blanket setting for every semiempirical calculation. An explicit `--weights` argument always layers on top.

11.3.3 A Dynamic Feature Vector

The feature vector is dynamic, not a fixed template. COSMIC reads whatever properties each quantum chemistry output actually contains and clusters on the **largest feature pool the dataset supports**: electronic and orbital energies, the HOMO–LUMO gap, dipole, nuclear repulsion, rotational constants, hydrogen-bond descriptors, and, when the calculation included frequencies, the Gibbs energy and vibrational frequencies. Any descriptor a structure lacks, or that carries no variance across the pool (the small-standard-deviation rule above), is left out automatically. The motifs are therefore always identified from the richest description available for that dataset, with no need for the user to declare in advance which properties exist.

11.3.4 Automatic Threshold Selection

The values in Table 7 are useful manual overrides, but by default COSMIC does **not** require the user to pick τ . In `automatic` mode the threshold is detected *per case* from the distribution of

merge heights in the UPGMA [11] linkage matrix. This matters because a single fixed value is not universally appropriate: for atomic clusters and weakly bound (dispersion- or van der Waals-dominated) systems the per-feature variance is intrinsically small, so almost every merge falls below $\tau = 2.0$ and physically distinct motifs collapse into a few mega-clusters. A data-driven cut fixes this.

What the heights mean (no factor of $\frac{1}{2}$). COSMIC builds the tree with SciPy’s average-linkage (UPGMA) routine and cuts it with `fcluster` on the *distance* criterion. Both the dendrogram height axis and every threshold — τ , the merge heights $h_{(i)}$ below, and the empirical 2.0 — are therefore expressed as the **full average inter-cluster Euclidean distance** of Eq. 5. Classical textbook UPGMA usually plots a node at *half* that value, the ultrametric branch length from the node down to its leaves; SciPy, and hence COSMIC, does **not** apply that factor of $\frac{1}{2}$. A merge height and τ are the inter-cluster distance itself — which is precisely why the $\tau = 2.0$ analysis below can be read directly in per-feature standard-deviation units.

Knee detection (default). Let $h_{(1)} \leq h_{(2)} \leq \dots \leq h_{(n-1)}$ be the sorted merge heights of the linkage matrix for n input structures ($n - 1$ merges). Plotted against merge index, this curve stays nearly flat while structures within the same motif fuse, then rises sharply once distinct motifs begin to merge; the elbow marks that transition and is the natural place to cut. Since the index and the height live on unrelated scales, both axes are first mapped to the unit interval:

$$\tilde{x}_i = \frac{i - 1}{n - 2}, \quad \tilde{y}_i = \frac{h_{(i)} - h_{(1)}}{h_{(n-1)} - h_{(1)}}, \quad i = 1, \dots, n - 1. \quad (15)$$

The perpendicular distance from the normalised point $(\tilde{x}_i, \tilde{y}_i)$ to the secant $\tilde{y} = \tilde{x}$ joining the first and last merges is

$$d_i = \frac{|\tilde{x}_i - \tilde{y}_i|}{\sqrt{2}}, \quad (16)$$

and the elbow is the merge of maximal deviation,

$$i^* = \arg \max_i d_i. \quad (17)$$

The cut is then placed *midway* between the elbow merge and the next one,

$$\tau_{\text{auto}} = \frac{1}{2}(h_{(i^*)} + h_{(i^*+1)}), \quad (18)$$

so the elbow merge itself is still treated as within-cluster and only merges in the steep regime act as separators. This is the standard maximum-distance-to-secant (“kneedle”) heuristic [12]; COSMIC uses a pure NumPy implementation to avoid an extra dependency.

Sanity gates and the $\tau = 2.0$ cap. The knee is trusted only when three conditions hold; otherwise COSMIC falls back to the empirical fixed threshold $\tau = 2.0$:

1. at least 8 merges ($n - 1 \geq 8$) — smaller datasets lack the curve structure for a reliable elbow;
2. a non-trivial dynamic range, $h_{(n-1)}/h_{(1)} \geq 5$ — otherwise the curve is essentially a straight line and the elbow is meaningless;
3. a sensible cluster count, $2 \leq k_{\text{auto}} \leq \lfloor n/2 \rfloor$ — guarding against one giant cluster or pathologically many.

In addition, the empirical value acts as an **upper limit**: if the detected knee exceeds 2.0, the threshold is capped at $\tau = 2.0$. The source of the finally applied threshold (**knee**, **legacy**, or **user**) is reported in the verbose log and on the diagnostic plot.

The empirical $\tau = 2.0$ value (the 2σ rule). This was the historical default and remains available as an explicit override (`--threshold 2.0`) and as the fallback above. Its justification is statistical. For two z -standardised feature vectors the squared Euclidean distance is $d^2 = n_c \overline{\Delta z^2}$, so the root-mean-square per-feature difference at the cut is $\Delta z_{\text{rms}} = \tau/\sqrt{n_c}$, measured directly in standard-deviation units:

Features (n_c)	Δz_{rms} at $\tau = 2.0$	Interpretation
5	0.89σ	per feature $\lesssim 1\sigma$
7	0.76σ	per feature $\lesssim 1\sigma$
11	0.60σ	per feature $< 1\sigma$
14	0.53σ	per feature $\approx 0.5\sigma$

Table 8: Per-feature RMS difference corresponding to $\tau = 2.0$ for typical feature-vector sizes n_c .

In every case $\tau = 2.0$ marks the boundary where the average per-feature difference crosses from *statistically indistinguishable* ($< 1\sigma$) to *statistically distinguishable* ($\geq 1\sigma$): structures merged below it share a common physicochemical profile within noise, while those separated above it differ meaningfully. The same boundary can be read as a Pearson similarity [13] between the two profiles, the basis of the per-cluster similarity floor reported by COSMIC.

Why Mojena is not used. The Mojena stopping rule [14] is a well-known clustering criterion and is still computed by COSMIC, but only as a diagnostic line on the validation plots — it is *not* part of the fallback chain. Across many independent test cases it proved too inconsistent to drive cluster selection reliably, which is why the knee method (with the $\tau = 2.0$ fallback) is preferred.

11.3.5 Running COSMIC

From a folder containing the QM outputs, launch the clustering with a chosen (or automatic) threshold and core count:

```
cosmic --th=auto -j4
```

COSMIC parses each output into the feature vector of Eq. 4, standardizes it, builds the UPGMA tree, and cuts it at the resolved threshold. Within each cluster the lowest-energy structure is kept as the representative motif. Adding `-rmsd` enables the optional Step II geometric check inside each family, which is illustrated in the case studies later in this manual.

Caching. Parsing hundreds of QM logs is COSMIC’s main bottleneck, so the parsed descriptors are cached to a `data_cache_XXXXXX.pkl` file on the first run. Re-clustering with a different threshold then reuses the cache instantly; pass `-reprocess-files` (or delete the cache) whenever the underlying outputs change.

11.4 Refinement of the Motifs

Once COSMIC has identified the unique motifs, they are refined at a higher level of theory (e.g. DFT) by the `ref` stage — the refinement counterpart of the `opt` pre-optimization. In the protocol it is written `ref input2`: it generates one high-level input per motif from the `input2` template, runs them, and hands the results to a second `cosmic` screening that yields the true unique minima and their Boltzmann populations. An *ultimate* run adds a third tier, `eref input3` — a high-level

single-point energy refinement (e.g. DLPNO-CCSD(T)) on the refined geometries — followed by one final `cosmic` pass.

12 Output Files

A successful run of ASCEC with the annealing option (“1” in Line 1) with candidate structure clustering using the COSMIC script for this example protocol will generate several files.

12.1 Print Levels: `miniprint` and `maxprint`

Every workflow runs at one of two print levels. **Miniprint** (the default) cleans up the bulky per-structure QM I/O at the end of the run, keeping only what is needed to interpret and reproduce the results; **maxprint** (`-maxprint`, the default for rigorous and ultimate modes) keeps every intermediate file from every stage. Miniprint typically shrinks a finished project from gigabytes to tens of megabytes.

Under miniprint each completed stage keeps its combined geometries (`combined_results.xyz/.mol`) and energy table (`*_summary.txt`), the full COSMIC analysis of each screening (dendrograms, `clustering_summary.txt`, `extracted_clusters/`, `extracted_data/`, and the `motifs_N/` or `umotifs_N/` representatives), the final ensemble, and `protocol_summary.txt`. The individual QM `.out` files, however, are retained *only for the motifs of the final, most expensive stage* — the DFT (or CCSD(T)) calculations a user may want to inspect for frequencies, orbitals or thermochemistry. A two-step GFN2→DFT run therefore reduces to:

```
project/
├── annealing/
├── optimization/
├── similarity/
├── refinement/
├── similarity_2/
├── geom_ref_out/
│   ├── umotif_01/
│   └── umotif_02/
├── final_ensemble.xyz
├── final_ensemble.mol
└── protocol_summary.txt
```

Here `optimization/` and `refinement/` keep only coordinates and summaries, while `geom_ref_out/` holds the complete ORCA files for each final motif. A run whose final stage carries no frequencies keeps a single `similarity/` stage and writes `possible_final_ensemble.*` instead; an ultimate GFN2→DFT→DLPNO-CCSD(T) run adds `refinement_2/` and `similarity_3/`, retaining `.out` files for both the geometry- and energy-refinement motifs. Unique-motif numbering is carried across stages, so a given `umotif_NN` denotes the same structure everywhere.

12.2 Annealing output

12.2.1 Main Log File (`w6.out`)

This file serves as the primary log, recording the complete history of the annealing protocol. To verify that the simulation completed successfully without runtime errors, users should inspect the final lines of the file (e.g., via the command `tail w6.out`). A successful run is confirmed by the presence of the termination flag:

```
** Normal annealing termination **
```

```
tail -n 15 w6_1.out
```

```
=====

** Normal annealing termination **
Total Wall time: 0 days 0 h 30 min 52 s 880 ms
Energy was evaluated 2919 times

Energy evolution in tvse_115889.dat
Configurations accepted by Max.-Boltz. statistics = 72
Accepted lower energy configurations = 55
Accepted configurations in result_115889.xyz = 127
Lowest energy configuration in rless_115889.out
Lowest energy = -0.51829172 u.a. (Config. 127)

=====
```

12.2.2 Reproducibility and Metadata

To ensure scientific rigor, ASCEC generates essential metadata for data validation and traceability. Each annealing run is initialized with a unique 6-digit random seed, which is used to seed all internal pseudorandom number generators (PRNGs). This mechanism ensures the stochastic independence of parallel runs, preventing statistical bias in the exploration of the potential energy surface. The seed is logged in the output file and appended to the names of key output files (e.g., `result_957775.xyz`, `tvse_957775.dat`), enabling clear identification and organization of results. Rerunning ASCEC with the same seed and input parameters will reproduce the same sequence of proposed molecular moves. However, exact reproduction of the full annealing trajectory additionally requires an identical software environment — the same Python and NumPy versions, as well as the same version, compilation, and parallelization settings of the external quantum-chemistry program (e.g., ORCA or xTB) — since numerical differences in energy evaluations, however small, can alter accept/reject decisions and cause trajectories to diverge.

Setting the seed. By default the 6-digit seed is drawn at random, but it can be fixed from the environment to reproduce a run. `COSMIC_ASCEC_SEED` sets a single seed; `COSMIC_ASCEC_SEEDS` takes a comma-separated list — one seed per replica — so an entire replicated protocol is reproducible from a single command:

```
COSMIC_ASCEC_SEED=878186 ascec glyw2.asc protocol
```

```
COSMIC_ASCEC_SEEDS=871246,112412,261557 ascec li5.asc protocol
```

The number of seeds in `COSMIC_ASCEC_SEEDS` must equal the number of replicas requested (three, for the `li5` example above); a mismatch is reported as an error. The same environment caveat applies: fixing the seed reproduces the sequence of proposed moves, but a bit-for-bit trajectory also needs the identical software stack described above.

12.2.3 Simulation Generated Data

I. Global Minimum Candidate (rless_XXXXXX.out)

This file contains the Cartesian coordinates corresponding to the lowest energy configuration visited during the entire annealing run. The suffix `XXXXXX` corresponds to the unique random seed of the run.

```
# Configuration: 141 | Energy = -71.65131472 u.a.
3
water1
O      3.114949    -2.572173    0.846802
H      3.740289    -3.084485    1.381054
H      2.849619    -3.185249    0.144893
3
water2
O      -1.728986     1.116733   -0.116378
H      -2.554571     0.817409   -0.525959
(...)
```

II. Energy Evolution Log (tvse_XXXXXX.dat)

This dataset records the thermodynamic history of the simulation. It is formatted in three columns: **Step Number**, **Temperature (K)**, and **Potential Energy (Hartree)**. Data is recorded exclusively for accepted Monte Carlo steps. This file is the source for generating the energy profile plots (see Figure 19) and the aggregate replica comparison (see Figure 20). These diagrams are automatically produced by the ASCEC script. To regenerate them manually, use the `diagram` command with the `--scaled` flag to auto-scale the Y-axis.

```
ascec diagram --scaled
```

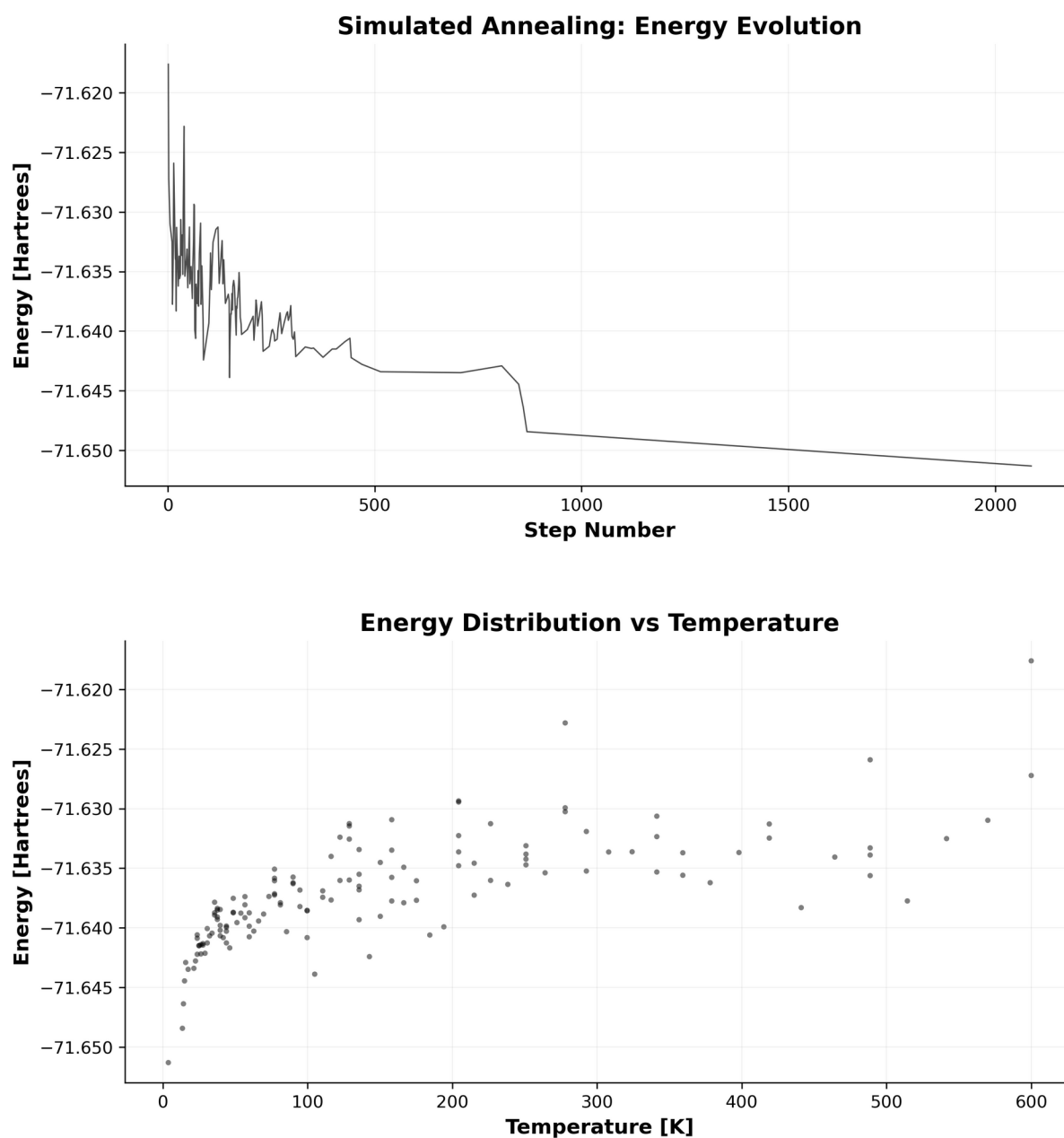



Figure 19: Single-run energy evolution profile (Temperature vs. Energy) generated by ASCEC.

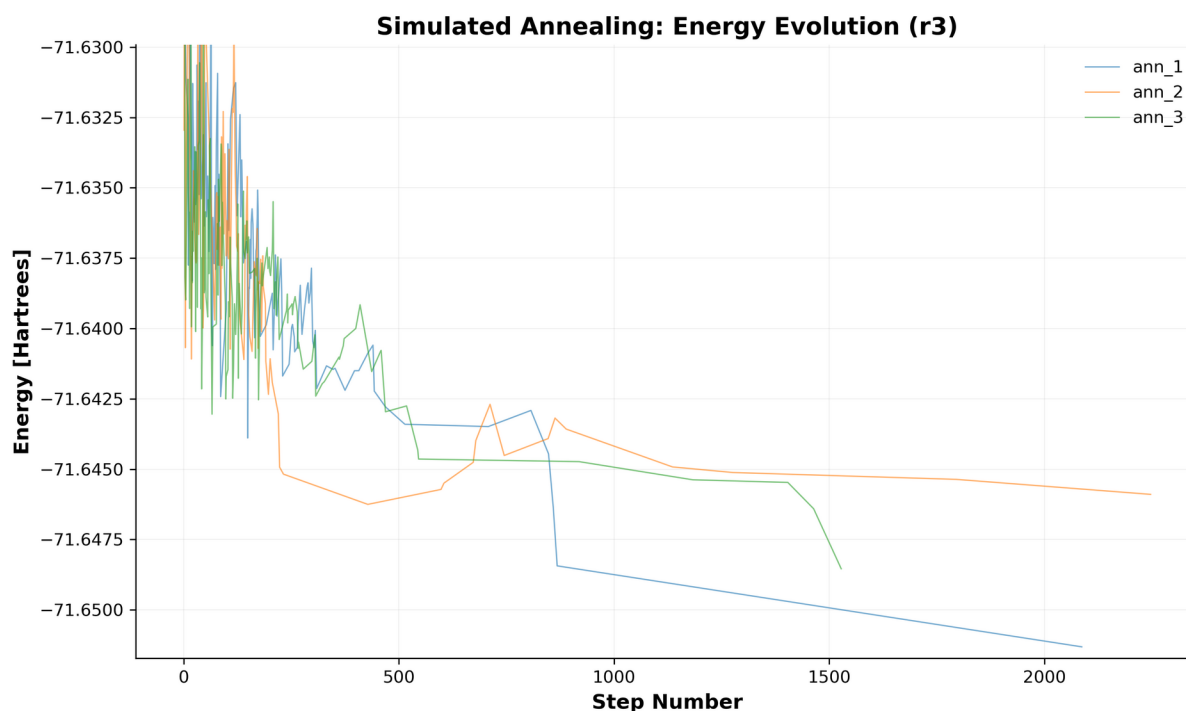


Figure 20: Combined energy profiles comparing multiple replica executions.

III. Structural Trajectory (result_XXXXXX.xyz | .mol)

These files contain the Cartesian coordinates of the entire ensemble of accepted structures. They serve as the full trajectory of the simulation, formatted for immediate visualization and animation using standard molecular editors (e.g., Avogadro2, Molekel, IQmol, or GaussView).

```

18
Configuration: 1 | E = -71.61760217 a.u. | T = 600.0 K
O      7.521687      8.123473      2.987348
H      7.729738      7.604909      3.779022
(...)
18
Configuration: 2 | E = -71.62722332 a.u. | T = 600.0 K
O      7.649736      8.853069      2.485145
H      8.312217      8.157098      2.610376
18
Configuration: 3 | E = -71.63098312 a.u. | T = 570.0 K
O      8.358924      7.910310      1.934682
H      9.138323      7.334893      1.954008

```

IV. Simulation Box Visualization (resultbox_XXXXXX.xyz | .mol)

Unless the `-nobox` flag is used, the program generates auxiliary files containing the simulation box vertices as dummy atoms. This allows for the visual verification of the confinement boundaries relative to the molecular system.

Compatibility Note:

- **Avogadro2:** Supports both `.xyz` and `.mol` formats for box visualization.
- **IQmol & GaussView:** Require the `.mol` file to correctly render the dummy atoms.

12.3 Calculation & Optimization Output

Once the process of calculating the generated input configurations and their sorting has been completed, the working directory (e.g., `calculation/` or `optimization/`) will typically contain the following key components:

- **Master Trajectories** (`combined_results.xyz` | `.mol`): Single master files containing the aggregated Cartesian coordinates for all processed configurations. These are generated to allow for the immediate visualization of the entire ensemble.

```

18
Coordinates from ORCA-job opt_conf_1 E -30.494666183120
O          4.89755049315488      6.57402652599668      3.11667076076455
H          4.49984221242737      6.65106539506547      4.01165830268415
(...)
18
Coordinates from ORCA-job opt_conf_2 E -30.489323286040
O          5.03567346983380      6.71360166724240      4.08623792011938
H          4.16925086054156      7.14879619512198      4.15686651181911
H          4.91370051289682      6.02838117620930      3.40385389606381

```

- **Job Subdirectories:** Dedicated folders containing the specific input/output files for each.
 - *In Calculation:* Naming depends on the input generation strategy:
 - * **Option ‘c’ (Combine):** `opt_conf_#` (e.g., `opt_conf_1`). Numbering is sequential across the merged dataset.
 - * **Option ‘a’ (Separate):** `opt#_conf_#` (e.g., `opt1_conf_1`). Explicitly identifies the source replica (e.g., Configuration 1 from Replica 1).
 - *In Optimization:* Named `motif_#` (e.g., `motif_01`).
- **Execution Summary** (`*_summary.txt`): A text file detailing the computational duration (wall time) and final energies for each calculation.
- **Launcher Script:** The shell script used to execute the batch jobs (e.g., `launcher_orca.sh`).

Exclusively in the **Calculation** directory, if the ****Combine**** strategy (Option c) was selected during setup, a file named `combined_r#` (e.g., `combined_r3`) is generated. This holds the raw merged data from the annealing replicas that were used to generate the initial input files.

12.4 COSMIC Output

Once the clustering process concludes, the results are stored in a dedicated directory. Note that the directory creation is driven by the `sort` command, which automatically increments the suffix (e.g., `cosmic_2`) to prevent overwriting. A typical output structure contains:

```

/
├─ dendrogram_images/
├─ extracted_clusters/
├─ extracted_data/
├─ motifs_53/
└─ orca_out_416/

```

```

|_ skipped_structures/
|_ boltzmann_distribution.txt
|_ clustering_summary.txt
|_ data_cache_105638.pkl

```

12.4.1 Dendrograms

The dendrogram is the main visualization tool for understanding the topology of the sampled set. It represents a hierarchical tree in which the vertical axis (Y) indicates the **Euclidean distance** (dissimilarity) between configurations.

By default, a single dendrogram (**dendrogram.png**) is generated for the entire dataset. The optional **--group-hb** flag produces per-H-bond-family dendrograms (**dendrogram_H{N}.png**), which can be useful for visual inspection of specific interaction networks. The examples below use **--group-hb** to illustrate family-specific trees.

Understanding the Clustering Logic (Figure 21) The structure of the dendrogram directly informs the choice of the clustering threshold (τ). As illustrated in the specific case of an H-Bond=10 family (using **--group-hb**), where three clusters were identified:

1. **Redundancy Identification:** Configurations **30** and **31** merge at a low height ($d < 1.0$). This implies they are geometrically and electronically equivalent.
2. **Cluster Differentiation (Intermediate):** Configuration **51** joins the (30+31) pair at a distance of $d \approx 3.5$. With a standard threshold of $\tau = 2.0$, this structure falls **above** the cutoff. Consequently, it is **not** grouped with the pair and is identified as a distinct motif.
3. **Global Separation:** Configuration **48** joins the ensemble at a much higher distance ($d \approx 5.8$). Since $5.8 \gg \tau$, it is recognized as a completely distinct basin of attraction, indicating significant structural and electronic differences.

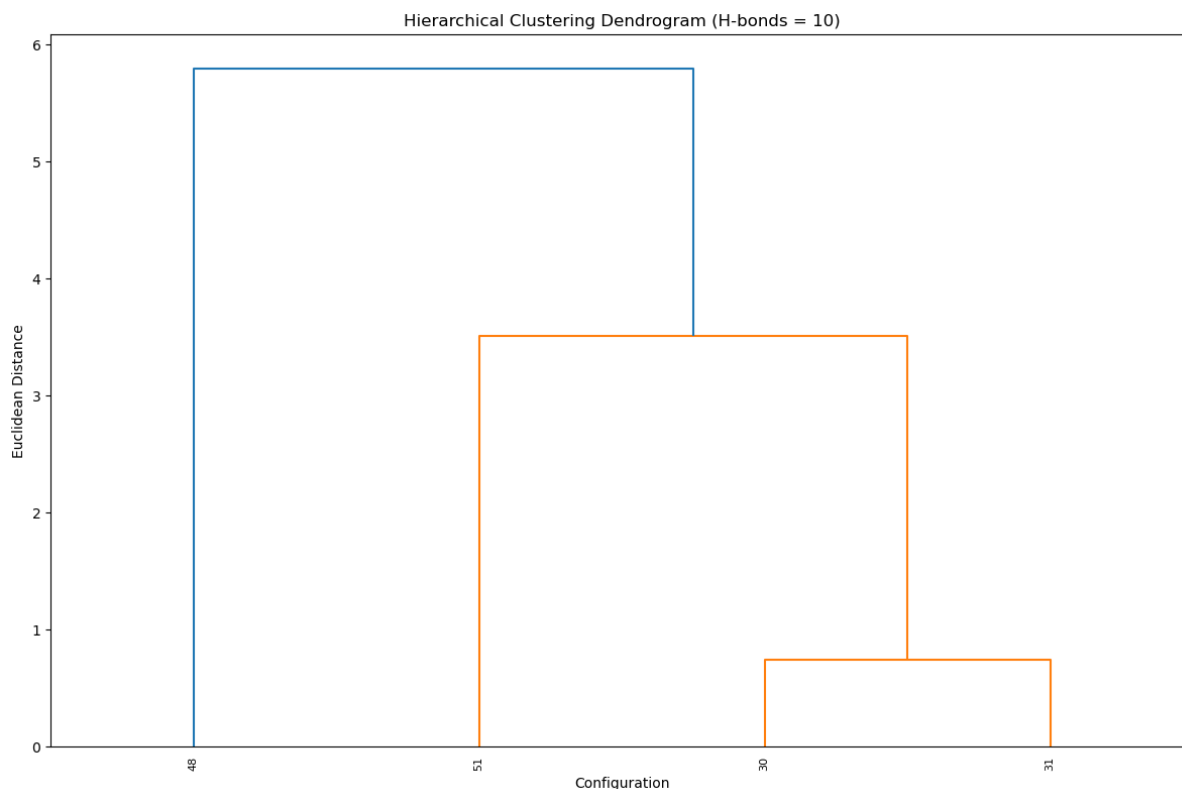


Figure 21: Detailed view of a dendrogram branch for the H-bonds=10 family (generated with `--group-hb`). The vertical height of horizontal nodes indicates the dissimilarity between branches.

Family-Specific Trees (`--group-hb`): When the `--group-hb` flag is active, the COSMIC module splits the dataset based on the number of detected Hydrogen Bonds (HB), generating a separate dendrogram for each subset. This allows granular inspection of specific interaction networks. Without this flag, all structures appear in a single unified dendrogram.

Pre-Optimization Analysis (GFN2-xTB) Figure 22 displays the clustering results for the H-Bond=9 family following the initial semi-empirical pre-optimization (using `--group-hb`).

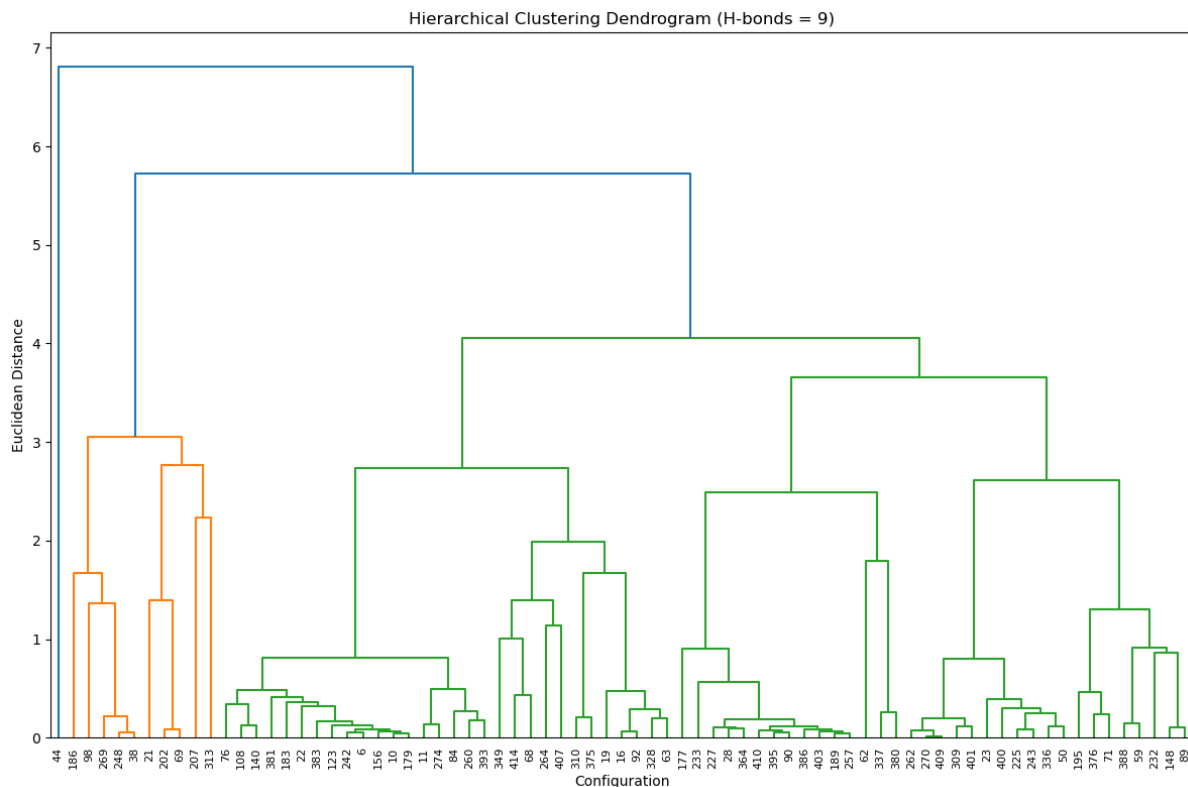


Figure 22: Dendrogram for the H-Bond=9 family following GFN2-xTB pre-optimization (generated with `--group-hb`).

As shown in Figure 22, the tree exhibits a high density of leaves with deep, complex branching. The presence of distinct major families at large Euclidean distances indicates that the annealing process successfully located fundamentally different structural basins. However, the dense grouping at the bottom of the clusters suggests a high degree of redundancy.

Hierarchical Reduction Strategy:

In this specific example, the preprocessing step successfully reduced the ensemble from **416 raw candidates** to **53 motifs**. These representatives were subsequently re-optimized using a higher level of theory (B97-3c), followed by a secondary clustering analysis which yielded **20 unique motifs**. It is crucial to note that the final number of motifs diverges from the intermediate count, as the topology of the Potential Energy Surface (PES) is sensitive to the electronic structure method employed; distinct semi-empirical minima may merge into a single basin upon DFT refinement. Furthermore, for this example, we did not correct the structures that the script flagged as critical and requiring manual inspection, so this final figure could be slightly higher.

- Impact of critical structures on total dataset: 8/416 configurations (1.9%)
- Impact of critical structures on total dataset: 4/53 configurations (7.5%)

Final Optimization Analysis (DFT/B97-3c) Figure 23 presents the topological classification of the H-Bond=7 family after high-level refinement.

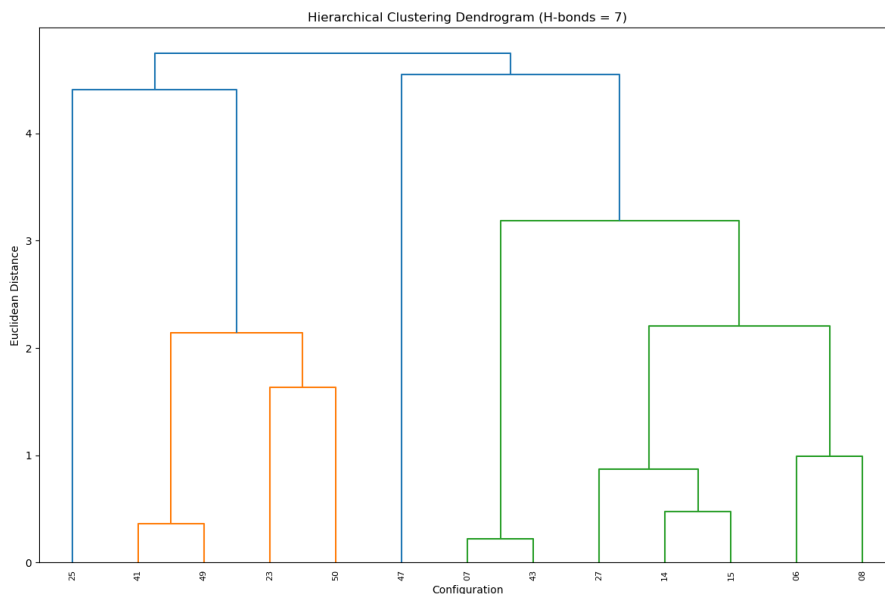


Figure 23: Dendrogram for the H-Bond=7 family following B97-3c final optimization (generated with `--group-hb`).

As shown in Figure 23, the topology is significantly more structured than the pre-optimization stage. The clear vertical separation between nodes confirms that the high-level optimization has relaxed the structures into well-defined, distinct minima, facilitating the selection of a robust threshold for the final Boltzmann population analysis.

Motifs Dendrogram Figure 24 presents the topological classification of the motifs found.

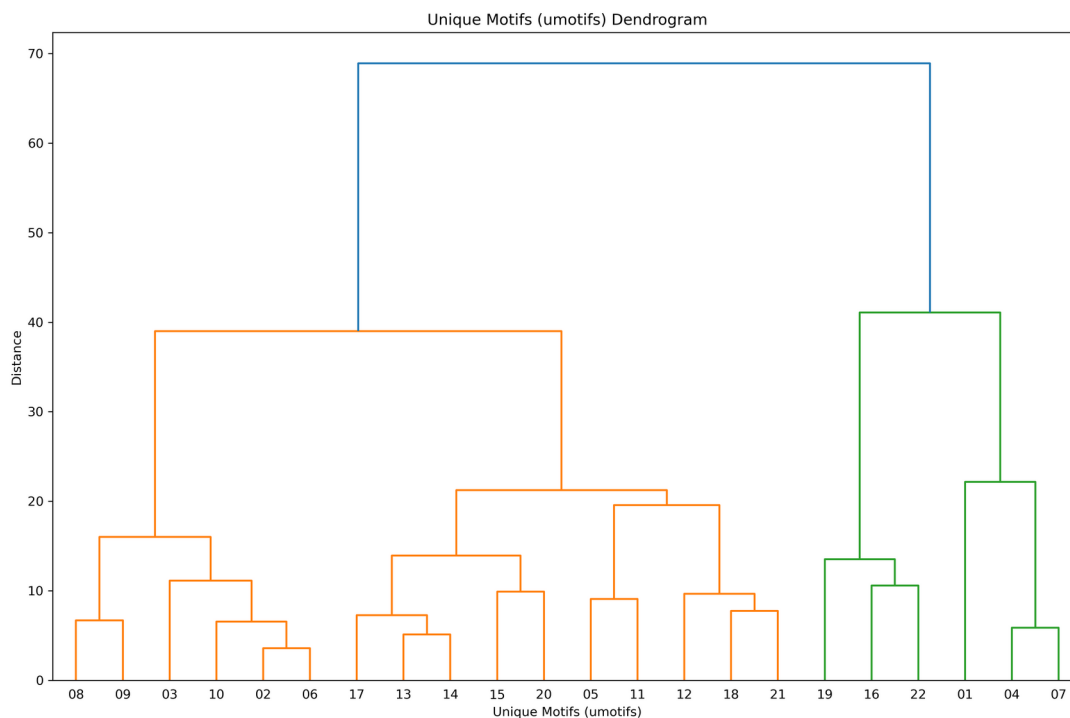


Figure 24: Dendrogram for the 22/23 unique motifs found after final optimization using B97-3c.

12.4.2 Clustering Summary Log

The script generates a comprehensive summary file (`clustering_summary.txt`) containing global statistics and detailed cluster breakdowns.

```
Clustering Results for: orca_out_416
COSMIC threshold (distance): 2.0
Total configurations processed: 416
Total files skipped: 23 (5.5%)
Critical skipped files: 8 (1.9%)
Total number of final clusters: 53
```

Example of Non-Converged Output When certain structures did not converge, the summary report indicates “N/A” for Gibbs energy. This indicates that the associated ORCA jobs reached the maximum number of optimization cycles without converging, so the thermochemical calculation step was not performed. Even so, the script was able to analyze the last Cartesian coordinates and attempted to cluster them.

```
All configurations

Configurations: 53
Number of clusters: 23

Cluster 22 (3 configurations):
Files:
- motif_25_opt.out (Gibbs Energy: N/A)
- motif_26_opt.out (Gibbs Energy: N/A)
- motif_31_opt.out (Gibbs Energy: N/A)

Cluster 23 (1 configurations):
Files:
- motif_42_opt.out (Gibbs Energy: N/A)
```

Valid Cluster Analysis For successfully converged systems, the summary provides precise thermodynamic data. Below is an example showing a clear separation between a singleton cluster (Cluster 6) and a grouping of two configurations (Cluster 5).

```
Cluster 5 (2 configurations):
Files:
- motif_23_opt.out (Gibbs Energy: -458.357020 Hartree (-287623.37 kcal/mol))
- motif_50_opt.out (Gibbs Energy: -458.356211 Hartree (-287622.87 kcal/mol))

Cluster 6 (1 configurations):
Files:
- motif_25_opt.out (Gibbs Energy: -458.356863 Hartree (-287623.27 kcal/mol))
```

Note: When using `--group-hb`, the summary groups results under “Hydrogen bonds: N” headers instead of “All configurations”.

12.4.3 Error Handling and Diagnostics

The COSMIC module implements a comprehensive diagnostic protocol to ensure ensemble integrity. It classifies files based on termination status, convergence, and vibrational topology:

1. Anomaly Classification The algorithm detects three specific types of problematic structures, which can be found in the output files:

- **Execution Failures:** Files lacking the "Normal Termination" flag (e.g., due to crash).
- **Non-Converged Geometries:** Files that did not reach a stationary point within the maximum optimization cycles despite normal termination.
- **Topological Instabilities:** Structures containing **one or more imaginary frequencies**. This includes first-order saddle points (Transition States), higher-order instabilities (Hilltops), or soft modes (e.g., unrelaxed methyl rotations).

2. Filtering Logic To balance data retention with redundancy reduction, anomalies are processed via clustering:

- **Skipped Files:** Files that cannot be parsed or failed execution are automatically skipped. Additionally, structures with imaginary frequencies are skipped if they cluster tightly ($d < \text{threshold}$) with a known true minimum, as these are considered "noisy" versions of a successfully identified basin.
 - **Flagged as Critical:** Any structure with imaginary frequencies that forms an **isolated cluster** (distinct from valid minima) is preserved and flagged as **Critical**. These are stored separately as they potentially represent:
 - * Unique Transition States (TS).
 - * Distinct local minimum that failed to converge purely due to numerical issues.
 - * Valid basin with a flat potential surface (soft modes).

Action Required: These files are exported for review and generally require manual inspection or re-optimization.

```

/
├─ skipped_structures/
│   └─ clustered_with_minima/
│       ├── motif_26_opt.out
│       ├── motif_26_opt.xyz
│       └─ need_recalculation/
│           ├── combined_need_recalc.xyz
│           ├── motif_11_opt.out
│           ├── motif_11_opt.xyz
│           └─ skipped_summary.txt

```

12.4.4 Generated Cluster Data

Beyond the summary, the script populates two key directories for detailed inspection:

1. **extracted_clusters/:** Contains subfolders for every identified cluster (e.g., **cluster_3_11** denotes Cluster #3 containing 11 configurations).

Inside, the individual or combined (**.xyz** | **.mol**) files allow for visual superposition and verification of the cluster.

2. **extracted_data/**: Contains statistical data files (e.g., **cluster_15_4.dat**). These text files list the raw physicochemical descriptors (dipole moments, rotational constants, energies) for every member of the group, enabling quantitative analysis of the cluster's homogeneity, along with weighting and tolerances.

Cluster (represented by: cluster_15_4) (4 configurations)

```
- opt_conf_239.out
- opt_conf_287.out
- opt_conf_322.out
- opt_conf_53.out
```

```
Deviation Analysis (Max-Min / |Mean|):
Gibbs Free Energy (Hartree) %Dev: 0.00%
HOMO-LUMO Gap (eV) %Dev: 0.04%
Dipole Moment (Debye) %Dev: 2.80%
Radius of Gyration (Å) %Dev: 0.21%
Rotational Constant A (GHz) %Dev: 0.49%
Rotational Constant B (GHz) %Dev: 0.49%
Rotational Constant C (GHz) %Dev: 0.62%
Last Vibrational Frequency (cm-1) %Dev: 0.02%
Number of Hydrogen Bonds %Dev: 0.00%
```

```
Clustering Weights Applied:
Gibbs Free Energy: 1.00
```

```
Clustering Absolute Tolerances Applied:
Gibbs Free Energy: 0.000005
```

12.4.5 Boltzmann Distribution

After optimization and clustering, the **boltzmann_distribution.txt** file is generated, ranking unique motifs by their Gibbs Free Energy at 298.15 K. While this reference temperature can be modified via the **-T=#** flag, the thermochemical properties must be recomputed accordingly to maintain accuracy. As illustrated below, the global minimum (UMotif 01) and the first local minimum (UMotif 02) are nearly iso-energetic, comprising more than 50% of the population.

```
Population by Energy Minimum
(assuming non-degeneracy, gi = 1)
```

```
Unique Motif (umotif) Assignment Summary
(sorted by Boltzmann population)
```

```
cluster_15 (umotif_01)
From structure: motif_03_opt
Gibbs Energy: -458.360815 Hartree (-287625.75 kcal/mol, -12472.63 eV)
Population: 26.77 %
```

```
cluster_1 (umotif_02)
From structure: motif_06_opt
Gibbs Energy: -458.360808 Hartree (-287625.75 kcal/mol, -12472.63 eV)
Population: 26.56 %
```

13 Workflow Mode

The automated protocol enables the execution of complex, multi-stage pipelines through a single directive. Specifically configured for the **ORCA** package. Unlike standalone execution—which requires user intervention to filter and resubmit files—the Workflow Mode autonomously manages the transition between annealing, optimization, and clustering. It includes built-in logic to correct problematic structures (e.g., imaginary frequencies), significantly reducing the manual overhead required to locate global minima.

13.1 Pipeline Logic

Figure 25 delineates the execution logic of the automated pipeline for a standard two-step refinement protocol. It illustrates the iterative progression from initial annealing to final refinement, including the automated feedback loops for correcting critical failures.

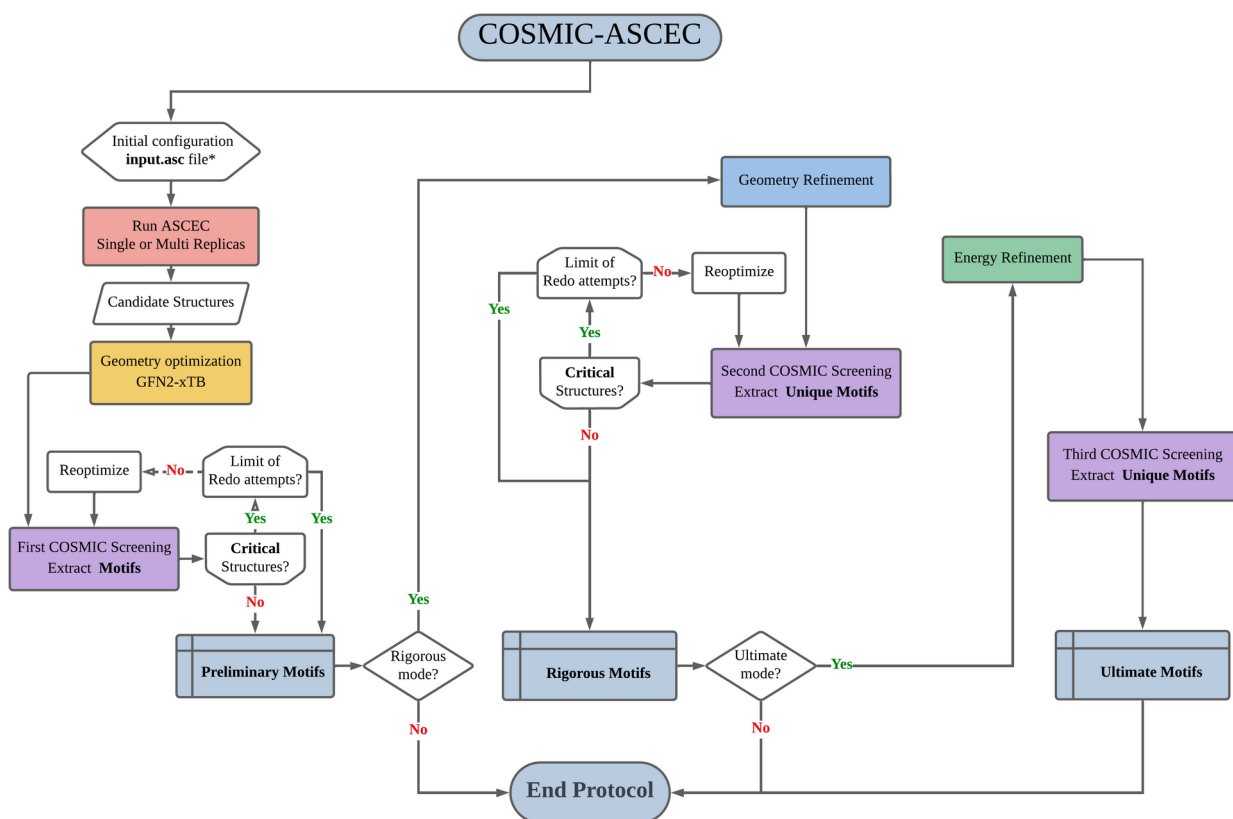


Figure 25: Schematic representation of the Two-Step Refinement Protocol.

13.2 Workflow Input (formic.asc)

To illustrate this functionality we reproduce the paper’s rigorous formic acid dimer run. Each monomer can adopt a *syn* or *anti* arrangement of its O–H and C–H bonds, so a dimer combines them as *aa*, *as* or *ss*. Starting from two superimposed *ss* (*syn*) monomers, the workflow runs the annealing and pre-optimization with the fast GFN2-xTB method (standalone xTB), then refines the COSMIC-filtered motifs at wB97X-D3BJ/def2-TZVPP with ORCA. The complete input below — parameters, coordinates, protocol, and the two embedded QM templates (Section 11.2.1) — is the only file required.

```
# Input for cis_formic Simulation
```

```

1 10          # Line 1: Simulation Mode & Number of Config
6.7          # Line 2: Simulation Cube Length (Angstroms)
2           # Line 3: Annealing Quenching route
100.0 10.0 50 # Line 4: Linear Quenching Parameters
500.0 5.0 100 # Line 5: Geometric Quenching Parameters
3000 10      # Line 6: Max Monte Carlo Cycles per T and floor value
1.0 1.0      # Line 7: Maximum Displacement (A) & Rotation (radians)
60 120       # Line 8: Conformational sampling (%) & Max dihedral (deg)
3 xtb        # Line 9: QM Program Index & alias (1:Gaussian 2:ORCA 3:xtb)
GFN2-xtb     # Line 10: Hamiltonian & Basis Set
1 4          # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1          # Line 12: Charge & Spin Multiplicity
2           # Line 13: Number of Molecules (nmo)

```

Lines below: Molecule Definition

```

*
5
cis1
C      0.340460   -0.288340   0.000000
O      1.570760   -0.288340   0.000000
O     -0.288840    0.913960   0.000000
H     -0.369640   -1.132440   0.000000
H     -1.252740    0.795160   0.000000
*
5
cis2
C      0.340460   -0.288340   0.000000
O      1.570760   -0.288340   0.000000
O     -0.288840    0.913960   0.000000
H     -0.369640   -1.132440   0.000000
H     -1.252740    0.795160   0.000000
*

```

Protocol

(comma = continue to next stage, dot = pause at breakpoint)

```

.asc,
--maxprint,
r5 --box20 --concurrent=5,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4

```

Embedded QM templates (Section labelled placeholders)

#xtb input1

#name

#rescue(GFN2-xtb/freq)

GFN2-xtb Opt Tight

nprocs 1

maxiter 200

#orca input2

#name

#rescue(HF-3c/freq)

! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3

```
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 80
end
* xyz 0 1
#
*
```

13.3 Protocol Command Syntax

The specific commands used in the **Protocol** section of the input file are detailed below.

Command / Flag	Description
<code>.asc</code>	Workflow Initiator. Designates the file as a multi-stage workflow script.
<code>-maxprint</code>	Retain I/O. Keeps every intermediate file from all stages (rigorous/ultimate default), instead of the space-saving miniprint .
<code>r5 -box20 -concurrent=5</code>	Annealing. Runs five ($r = 5$) independent replicas concurrently. <code>-box20</code> sizes the box for 20% effective packing (rigorous default).
<code>opt -concurrent=8 input1</code>	Pre-optimization. Optimizes every accepted structure with the embedded <code>input1</code> template (GFN2-xTB), 8 jobs at a time.
<code>cosmic -j4</code>	First screening. Clusters the pre-optimized pool with the automatic (knee) threshold on 4 cores, keeping one motif per family.
<code>ref -concurrent=2 input2</code>	Refinement. Re-optimizes the motifs at high level with the embedded <code>input2</code> template (wB97X-D3BJ/def2-TZVPP Opt+Freq), 2 jobs at a time.
<code>cosmic -th=opt -j4</code>	Final analysis. Re-uses the threshold resolved by the first screening (<code>-th=opt</code>) to cluster the refined structures into the unique minima and their Boltzmann ensemble.

Table 9: Commands and flags used in the rigorous formic acid workflow. The convergence-tolerance flags (`-critical`, `-skipped`, `-retry`, `-redo`) are optional; their defaults, described in the Command Reference, drove the automatic error correction in this run.

13.4 Execution and Outputs

The automated protocol is initiated via the following command:

```
ascec formic.asc protocol
```

13.4.1 Prerequisites and File Structure

Prior to execution, specific configuration and template files must be present in the working directory. These templates must adhere to the syntax defined in Section 11 (see 11.2.1), utilizing specific placeholders for dynamic data injection:

- **#name:** (Optional) Replaced by the specific filename to ensure traceability in output logs.
- **#:** Indicates the insertion point for the Cartesian coordinate block.
- **###:** Designates the location within the launcher script where serial execution commands will be injected.

Upon initialization, the system parses these files and generates a structured directory tree for each stage of the workflow.

Annealing → Calculation → COSMIC → Optimization → COSMIC(2)

Required Input Files	Generated Output Structure
<pre> / ├─ formic.asc ├─ launcher_orca.sh ├─ opt_wb97x.inp └─ preopt_gfn2.inp </pre>	<pre> / ├─ annealing/ ├─ calculation/ ├─ optimization/ ├─ cosmic/ ├─ cosmic_2/ ├─ protocol_425319.pkl └─ protocol_summary.txt </pre>

13.4.2 Terminal Output

When the workflow is initiated, the system parses the protocol and begins the first stage:

```

Protocol mode activated
Creating new protocol cache: protocol_670892.pkl (for cis_formic.asc)
Resuming workflow

Workflow: annealing → geometry_optimization → cosmic → geometry_refinement →
cosmic

-----
[1/5] Annealing
-----
Using recommended box size: 6.7 Å (20.0% effective packing)
Creating 5 replicated runs in 'annealing/'...
Created: formic_1/formic_1.asc
Created: formic_2/formic_2.asc
Created: formic_3/formic_3.asc
Created: formic_4/formic_4.asc
Created: formic_5/formic_5.asc
Running 5 annealing simulation(s)

formic_1...
```

13.5 Execution Control and Resumption

The workflow allows for granular control over the execution pipeline via delimiters and caching.

13.5.1 Stage Delimiters

- **Continuous Execution (,):** A comma at the end of a protocol line indicates that the script should proceed immediately to the next stage upon completion.
- **Breakpoints (.):** A period (or dot) acts as a breakpoint. The workflow will complete the current stage and then pause, allowing the user to manually inspect intermediate files (e.g., to review **Critical** structures).

13.5.2 Resuming the Workflow

The system maintains a cache file (e.g., `protocol_425319.pkl`) to track progress. If a workflow is paused via a breakpoint or interrupted, it can be resumed from the last successful stage using the same command:

```
ascec formic.asc protocol
```

This ensures that expensive calculations are not repeated unnecessarily.

Resuming the Pipeline

Users can direct the workflow to start from a specific stage index:

```
1.Annhealing → 2.Calculation → 3.COSMIC → 4.Optimization → 5.COSMIC2
```

Force Restart: To start over from the beginning of the Calculation stage (2):

```
ascec formic.asc protocol 2
```

Resume Incomplete: To resume the Calculation stage (2) preserving valid progress:

```
ascec formic.asc protocol 2 -i
```

13.6 Failure Recovery Mechanisms

The workflow incorporates robust fault-tolerance mechanisms to handle the complex potential energy surfaces often encountered in difficult systems.

13.6.1 Retry Logic (Execution Reliability)

This protocol addresses execution failures where a calculation does not achieve "Normal Termination" (e.g., non-zero exit codes or crashes). The script attempts to recover using one of two strategies: restarting the calculation from scratch or resuming from the last recorded coordinates. This feature is designed to mitigate transient execution errors, including filesystem disruptions and operational noise. It cannot correct systematic errors arising from incorrect input syntax; therefore, the ORCA input templates must be pre-validated for the system under study.

```
-----  
[2/5] Calculation  
-----
```

```
Executing calculations...
```

```
Running: opt_conf_1.inp... ✓ (2nd Attempt)
```

```
Running: opt_conf_2.inp... ✓
```

```
...
```

```
Running: opt_conf_495.inp... ✓
```

```
Running: opt_conf_496.inp... ✓
```

```
Status: 496/496 calculations completed
```

```
All calculations completed successfully
```


13.6.2 Redo Logic (Convergence and Topology)

The final ensemble must consist exclusively of **true local minima**. To ensure no optimized structure retains imaginary frequencies, the Redo logic automatically intervenes when topological instabilities are detected:

1. **Multiple Imaginary Frequencies:** Treated as numerical convergence artifacts. The system attempts to resolve these by re-submitting the optimization using the final coordinates of the previous run.
2. **Single Imaginary Frequency:** Treated as first-order saddle points (Transition States) or soft vibrational modes. The algorithm **perturbs the geometry along the imaginary vibrational mode** to push the structure toward a stable basin before re-optimizing.

Computational Cost Warning: Since this process involves high-level refinement, it is computationally expensive. It is intended to resolve soft modes and saddle points, not to compensate for an inappropriate method.

```
-----
[4/5] Optimization
-----
Stage redo enabled: max 10 attempts, target skipped ≤ 0.0%

Executing optimization calculations...
Running: motif_01_opt.inp... ✓
...
Running: motif_35_opt.inp... ✓

Status: 35/35 optimizations completed
Optimization calculations completed

-----
[5/5] COSMIC
-----
Using cosmic script: cosmic-v01.py

Working directory: cosmic_2

Processed 2 structures with imaginary frequencies:
- 0 clustered with true minima (can be ignored)
- 2 may represent missing motifs (need recalculation)
  motif_25_opt.out - 1 imaginary freq(s)
  motif_12_opt.out - 2 imaginary freq(s)

Finished processing folder: orca_out_35
All selected molecular analyses complete!

Results: 5.7% critical, 5.7% skipped
→ Threshold exceeded (skipped 5.7% > 0.0%)
```

Clarification of Metrics: The **Critical** set is a specific subset of the **Skipped** set. While all structures containing imaginary frequencies are automatically classified as **Skipped**, the **Critical** designation is reserved exclusively for those that form **isolated motifs**.

```

-----
Redo attempt 2/10

Processing redo structures from: cosmic_2/skipped_structures
Found 2 structure(s) to retry

motif_12_opt: 2 imaginary freqs, using final geometry ✓
motif_25_opt: 1 imaginary freq, displacing along mode ✓

Regenerated 2 input file(s) with new geometries
Resuming: Using existing optimization directory (Redo Mode)

Executing optimization calculations...
Resuming: 33/35 optimizations already completed
Running: motif_12_opt.inp... ✓
Running: motif_25_opt.inp... ✓

Status: 35/35 optimizations completed

Using cosmic script: cosmic-v01.py
python ../../cosmic-v01.py --th=2 -j4 --update-cache /tmp/tmpk17rmd.txt

```

14 Results and Validation

This section presents validation case studies demonstrating the efficacy and robustness of the COSMIC ASCEC protocols. We evaluate the software across three distinct dimensions:

1. **Standalone Execution:** A hierarchical Two-Step protocol using the Water Hexamer (H_2O)₆ [10].
2. **Automated Workflow:** A demonstration of the fully automated pipeline utilizing the *cis*-Formic Acid Dimer [15].
3. **Clustering Algorithms:** An assessment of standard topological clustering versus RMSD-refined hierarchical clustering.

14.1 Standalone Execution: Water Hexamer

To validate the modular utility of the software, the Water Hexamer system (**w6.asc**) was analyzed using a manual execution strategy. This case study benchmarks the trade-off between computational cost and configurational space coverage.

14.1.1 Hierarchical Two-Step Protocol (ORCA)

This protocol employed a cost-effective pre-optimization filter using the semiempirical GFN2-xTB method, followed by high-level DFT refinement. The workflow processed an initial stochastic pool of **416 raw annealing candidates**.

1. Geometric Reduction (GFN2-xTB): The COSMIC module reduced the raw input of 416 structures to ****53 representative configurations**** based on topological distinctness. Figure 26 illustrates the ensemble members of a specific cluster (**cluster_1_24**), while Figure 27 displays its representative structure.

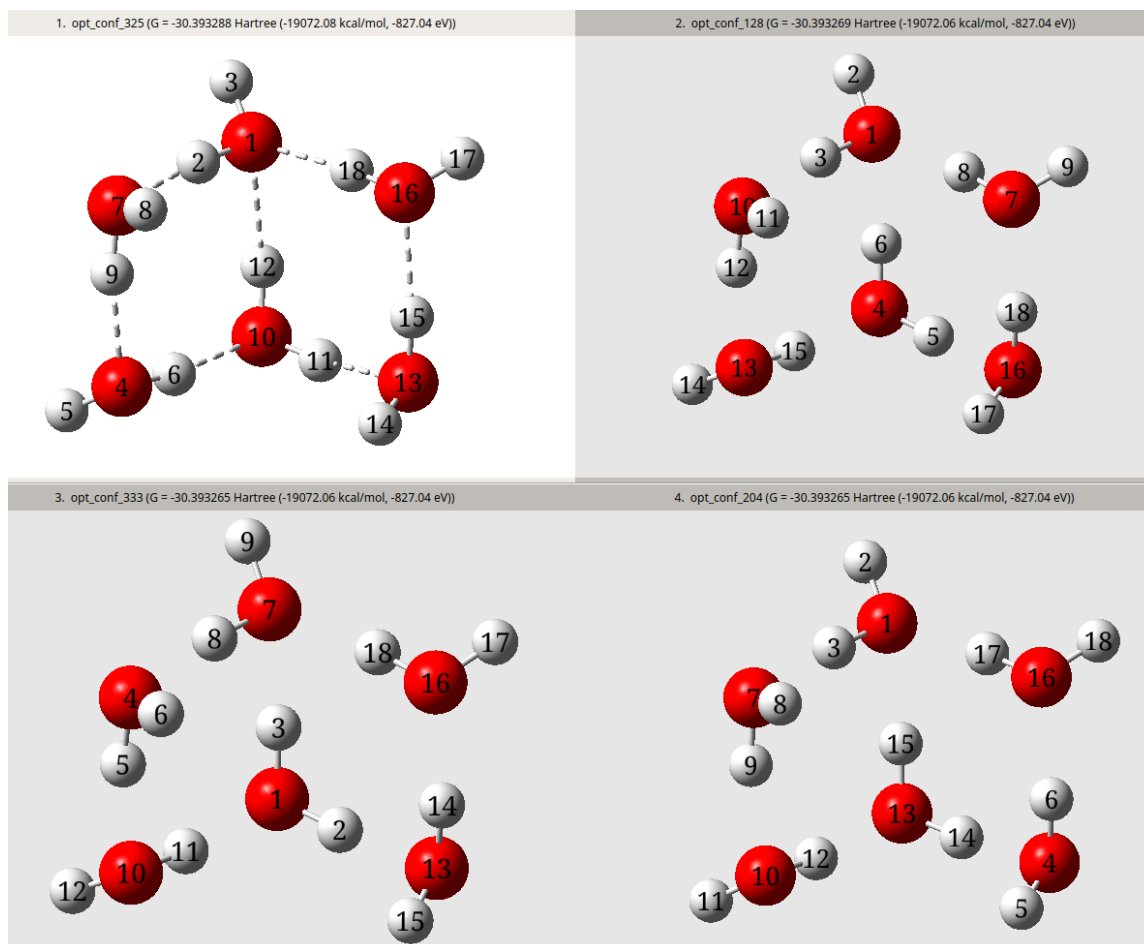


Figure 26: Visual inspection of the `cluster_1_24` ensemble members.

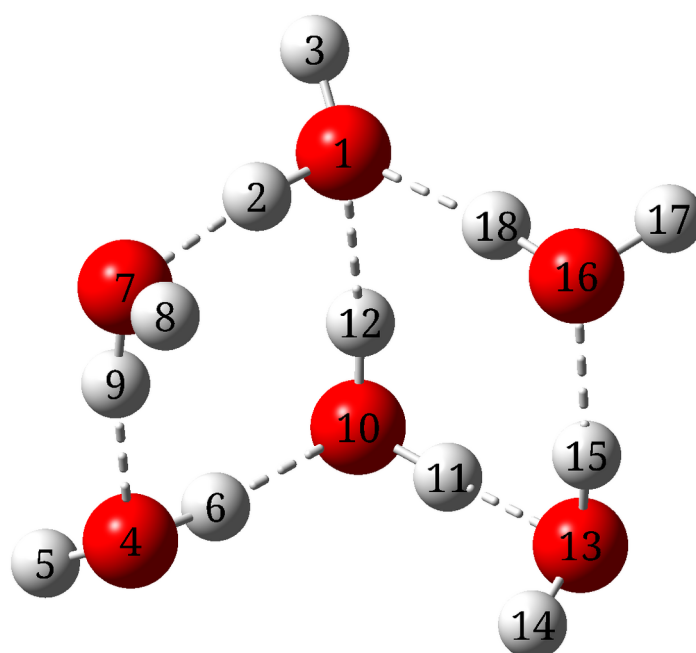


Figure 27: Representative configuration for `cluster_1_24`.

2. High-Level Refinement (B97-3c): The 53 representatives underwent full DFT optimization (B97-3c). A final COSMIC check (Threshold $\tau = 2.0$) consolidated these into ****20 Unique Motifs****. Some of them can be seen in Figure 28. In addition, Figure 29 highlights the three most statistically significant structures.

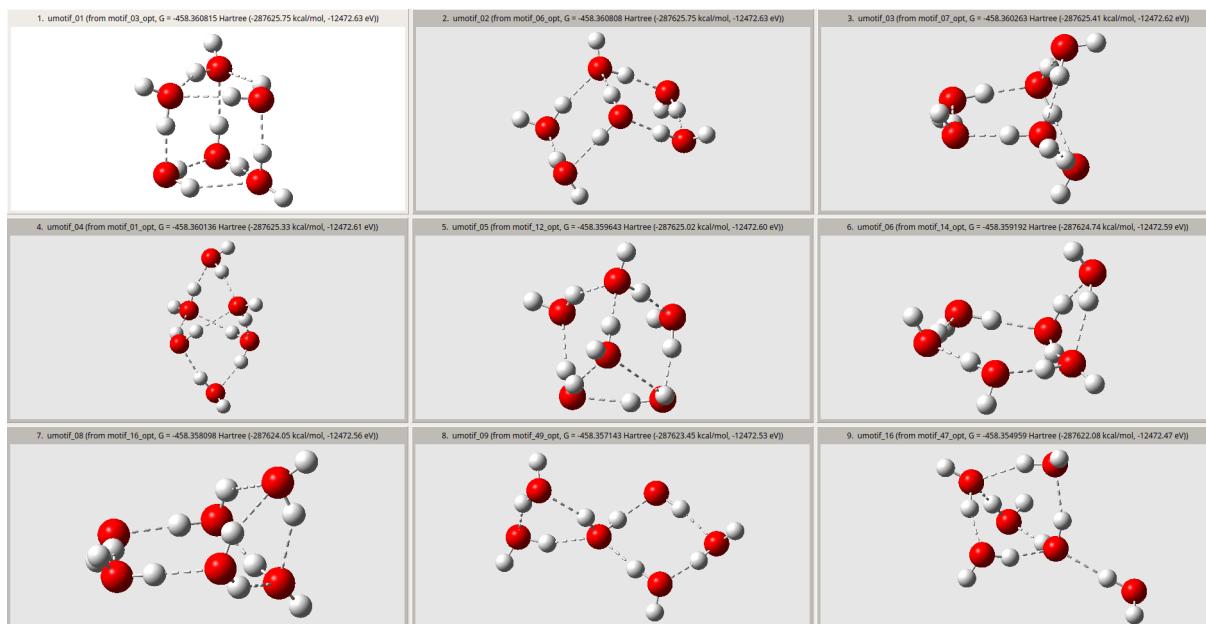


Figure 28: Subset of unique motifs identified for $(H_2O)_6$ system via the Two-Step protocol.

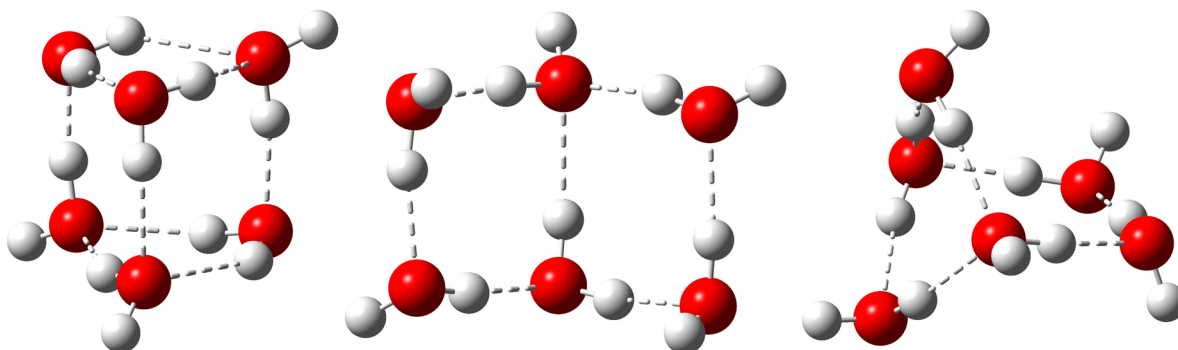


Figure 29: The three dominant water hexamer motifs identified by ASCEC.

Boltzmann Distribution: Thermodynamic analysis at 298.15 K indicates that the top three motifs account for over **68%** of the total ensemble population. The global minimum (Prism-like, UMotif 01) and the first local minimum (Book-like, UMotif 02) are effectively iso-energetic.

Unique Motif (UMotif) Assignment Summary
(Sorted by Boltzmann population - 298.15 K)

UMotif_01 (cluster_15)
Gibbs Energy: -458.360815 Hartree (-287625.75 kcal/mol)
Population: 26.35 %

```
UMotif_02 (cluster_1)
Gibbs Energy: -458.360808 Hartree (-287625.75 kcal/mol)
Population: 26.14 %

UMotif_03 (cluster_2)
Gibbs Energy: -458.360263 Hartree (-287625.41 kcal/mol)
Population: 14.69 %
```

14.1.2 Direct Approach (Gaussian 09) & Comparative Analysis

The Gaussian 09 validation employed a direct optimization strategy at the wb97XD/6-31+G(d) level of theory, skipping the semiempirical pre-screening. Comparing the Hierarchical and Direct method approaches reveals critical insights regarding Potential Energy Surface (PES) sampling and computational efficiency.

1. Topological Accuracy and Semiempirical Bias Both methods successfully identified the dominant low-energy isomers (Prism, Cage, Book). However, a detailed inspection of the full ensembles reveals that the semi-empirical pretreatment (GFN2-xTB) exhibits a specific topological bias that must be taken into account.

GFN2-xTB tends to over-stabilize cooperative hydrogen bonding networks, prioritizing structures where water molecules act as both donors and acceptors. Consequently, configurations containing "pure acceptor" water molecules (which do not donate hydrogen) are occasionally filtered out during the pre-optimization step. Figure 30 illustrates such a structure, which was found by the direct approach but excluded by the two-step protocol.

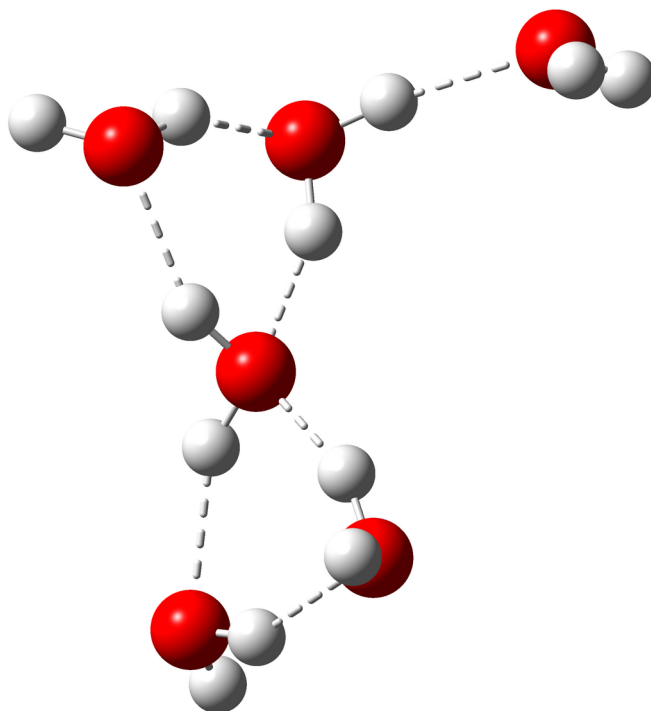


Figure 30: A specific water hexamer configuration identified only by the Direct Approach (Gaussian 09), characterized by a pure-acceptor water molecule.

Despite this topological divergence, the thermodynamic impact is negligible. The missing structure possess high relative energies and contribute $\approx 0\%$ to the Boltzmann distribution at 298.15 K. Furthermore, comparisons with high-level benchmarks (Figure 31) confirm that the ASCEC Two-Step protocol structures closely replicate the MP2/6-311++G** geometries reported by [10], in cases where the presence of pure-acceptor water molecules is **not mandatory**.

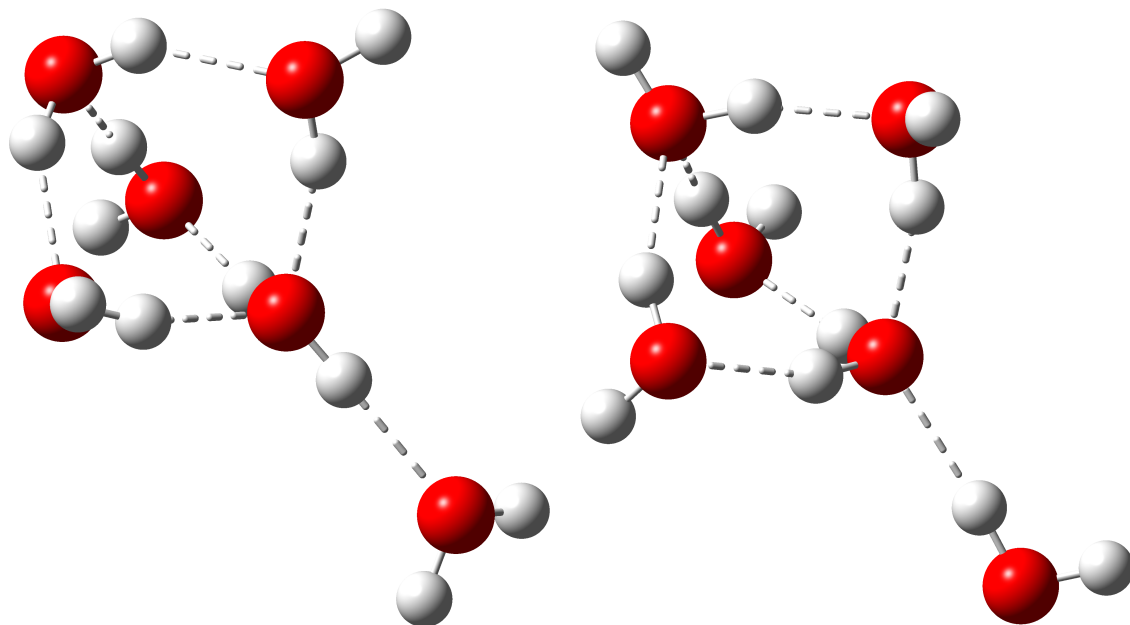


Figure 31: Structural comparison of the MP2/6-311++G** reference geometry (Left) [10] and the ASCEC-derived GFN2-xTB//B97-3c structure (Right).

2. Computational Efficiency The decisive advantage of the hierarchical two-step protocol is cost. Because the raw annealing pool is first collapsed to a handful of representatives by the GFN2-xTB pre-optimization and the first COSMIC pass, only those few structures reach the expensive DFT stage. The direct approach instead sends every candidate straight to DFT, so it pays the high-level cost hundreds of times over for a final ensemble that, as shown above, is essentially identical in its thermodynamically relevant minima. The two-step route therefore delivers the same PES at a small fraction of the wall time.

14.2 Automated Workflow: Formic Acid Dimer

To demonstrate the autonomy of the ASCEC pipeline, the Formic Acid Dimer system was processed using the **Workflow Mode**, following the rigorous protocol of the accompanying paper. Unlike the manual execution, this protocol autonomously managed error correction, imaginary frequency elimination, and convergence criteria to recover the full *aa/as/ss* dimer family, and to locate the cyclic global minimum, starting from two superimposed *ss* (*syn*) monomers (Figure 32).

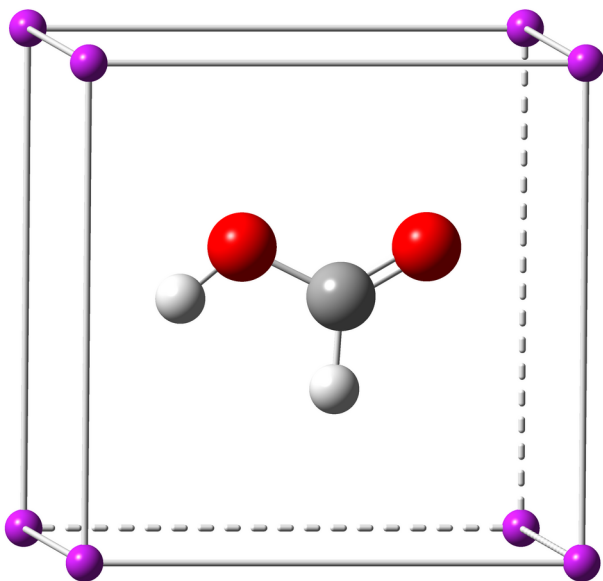


Figure 32: Initial simulation box containing *cis*-Formic Acid monomers.

14.2.1 Protocol and Computational Efficiency

The rigorous run used five independent annealing replicas, pre-optimized every accepted structure at GFN2-xTB (standalone xTB), and refined the COSMIC-filtered motifs at wB97X-D3BJ/def2-TZVPP with full frequency analysis (ORCA). Starting from the **6.7 Å** box, the five replicas accepted **574 candidates**, and the complete pipeline finished in **10 h 43 min**.

Consistent with every other benchmark, the stochastic search is computationally negligible. The recorded `protocol_summary.txt` splits the wall time as annealing **0.1%**, GFN2-xTB pre-optimization **0.2%**, and DFT refinement **99.7%** (Figure 18): almost the entire cost is the high-level stage acting on the handful of structures COSMIC retained.

14.2.2 Automated Error Correction and Robustness

The two COSMIC screenings behaved very differently, which is instructive. After the GFN2-xTB pre-optimization the first screening reduced the 574 structures to **46 motifs** with **0% critical** and **0% skipped**: at the semiempirical level every retained structure was already a clean minimum, so no redo was triggered.

The picture changed once the 46 motifs were refined at wB97X-D3BJ/def2-TZVPP. The final screening initially found **11 of them (23.9%)** carrying imaginary frequencies — soft modes and near-saddle points that the tighter DFT surface exposes. Because the protocol requires a clean final ensemble (`critical` $\leq 0\%$), the **Redo** loop engaged automatically:

- a structure with a *single* imaginary frequency is displaced along that mode and re-optimized;
- a structure with *several* imaginary frequencies is re-optimized from its last geometry, treating them as convergence artifacts.

Over **three redo attempts** all 11 flagged structures were recovered as genuine minima, bringing the critical fraction to **0%** (Figure 33). Structures that survive every redo attempt are usually not minima at all but dissociated arrangements — fragments driven far apart during the search (> 7.5 Å, Figure 34) — which the algorithm correctly refuses to count; in this run none remained.

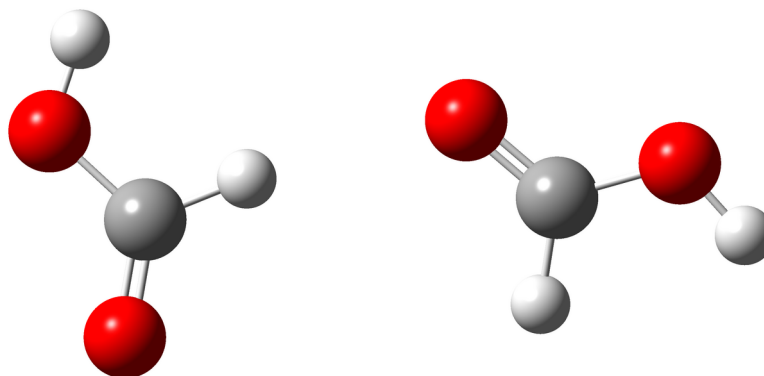


Figure 33: A structure flagged with an imaginary frequency and automatically recovered as a valid minimum by the Redo loop.

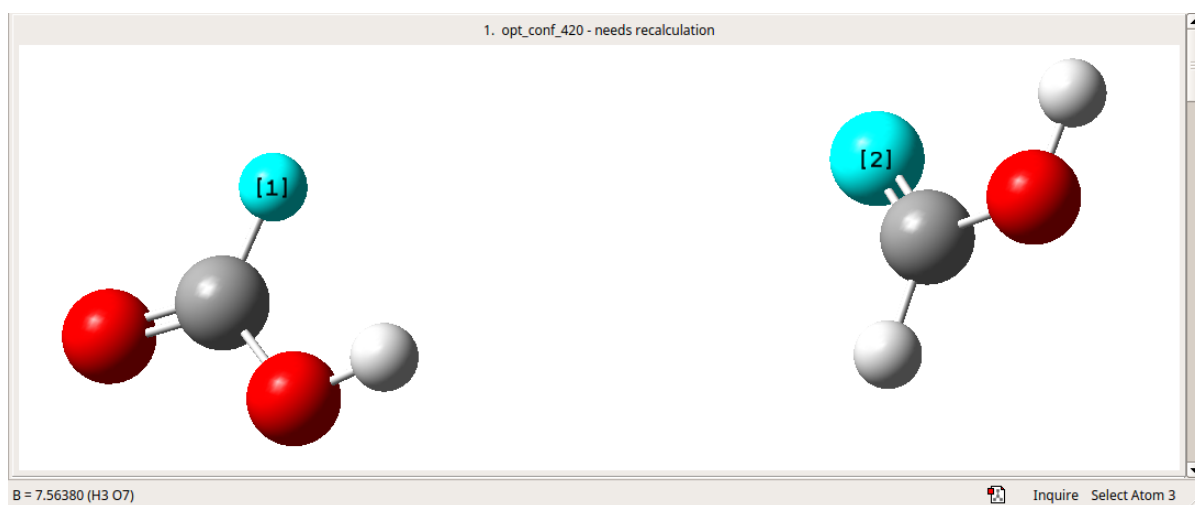


Figure 34: A dissociated arrangement ($r > 7.5 \text{ \AA}$) of the kind flagged as *critical* when it cannot be recovered; such structures are excluded from the final ensemble.

14.2.3 Thermodynamic Analysis

The final COSMIC pass reused the automatic (knee) threshold resolved by the first screening — this is what `-th=opt` means — and consolidated the 46 refined structures into **21 unique motifs** (7 *aa*, 10 *as*, 4 *ss*; one visual redundancy remains), of which 18 coincide with the reference MP2 calculations of the literature (Figure 35). The Boltzmann distribution at 298.15 K is overwhelmingly dominated by the cyclic *aa* dimer: its double hydrogen bond holds **99.99%** of the population, and the next motif already lies **5.46 kcal/mol** higher, so every remaining conformer is effectively inaccessible at room temperature ($RT \approx 0.59 \text{ kcal/mol}$).

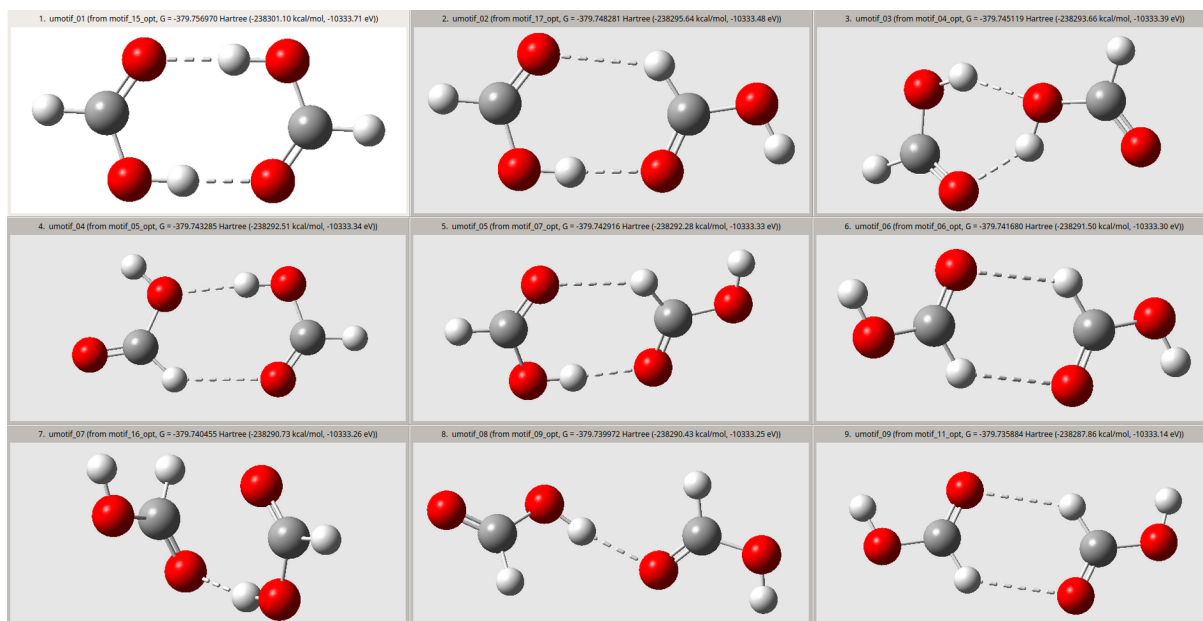


Figure 35: Subset of unique motifs identified for the formic acid system.

The global minimum (UMotif 01, Figure 36) corresponds to the cyclic *aa* dimer with a double hydrogen bond. Remarkably, although the input used two *ss* (*syn*) monomers, the search recovered the complete *aa/as/ss* family and correctly identified the cyclic *aa* structure as the global minimum, demonstrating the protocol's ability to overcome the monomer isomerization barrier during the stochastic search.

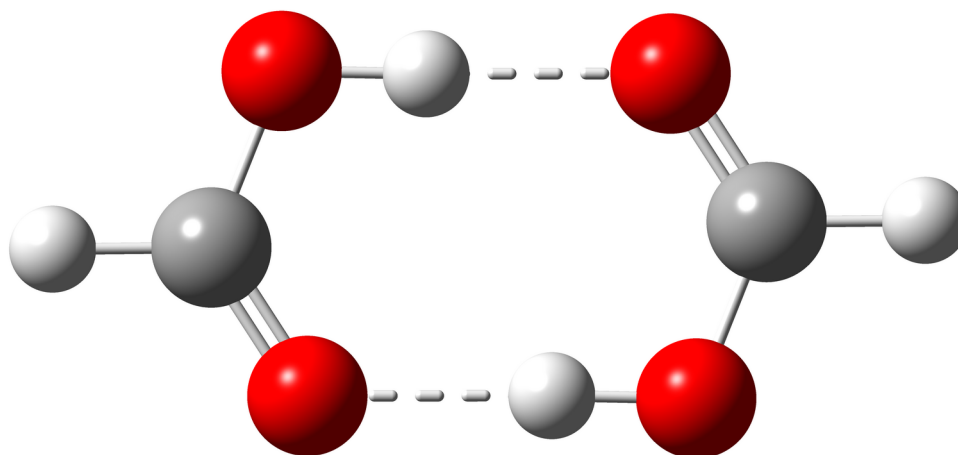


Figure 36: Global minimum: The cyclic Formic Acid Dimer.

Unique Motif (umotif) Assignment Summary
(sorted by Boltzmann population)

```
cluster_1 (umotif_01)
From structure: motif_21_opt
Gibbs Energy: -379.756973 Hartree (-238301.10 kcal/mol, -10333.71 eV)
Relative Gibbs Energy: 0.000000 Hartree (0.00 kcal/mol)
Population: 99.99 %
```

```
cluster_2 (umotif_02)
From structure: motif_02_opt
Gibbs Energy: -379.748280 Hartree (-238295.64 kcal/mol, -10333.48 eV)
Relative Gibbs Energy: 0.008694 Hartree (5.46 kcal/mol)
Population: 0.01 %

cluster_3 (umotif_03)
From structure: motif_03_opt
Gibbs Energy: -379.745063 Hartree (-238293.62 kcal/mol, -10333.39 eV)
Population: 0.00 %
```

14.3 Clustering Methodologies: Topology vs. Geometry

This section evaluates the impact of the hierarchical clustering protocols on ensemble generation. We contrast the coarse-grained Physicochemical Feature Vector (Step I) approach against an fine-grained RMSD refinement (Step II) hierarchical strategy to determine the optimal use cases for either and to decide how to choose between them.

14.3.1 Case Study A: Resolving Spatial Isomerism (Adenine-Thymine)

The limits of purely vector-based clustering (Step I) are evident in systems where distinct stereoisomers possess identical internal connectivity and energies. In the Adenine-Thymine dimer, the standard algorithm (Threshold $\tau = 2.0$) grouped spatially distinct configurations into a single family (`cluster_17`). While physicochemical properties are conserved, the spatial orientation differs significantly (Figure 37).

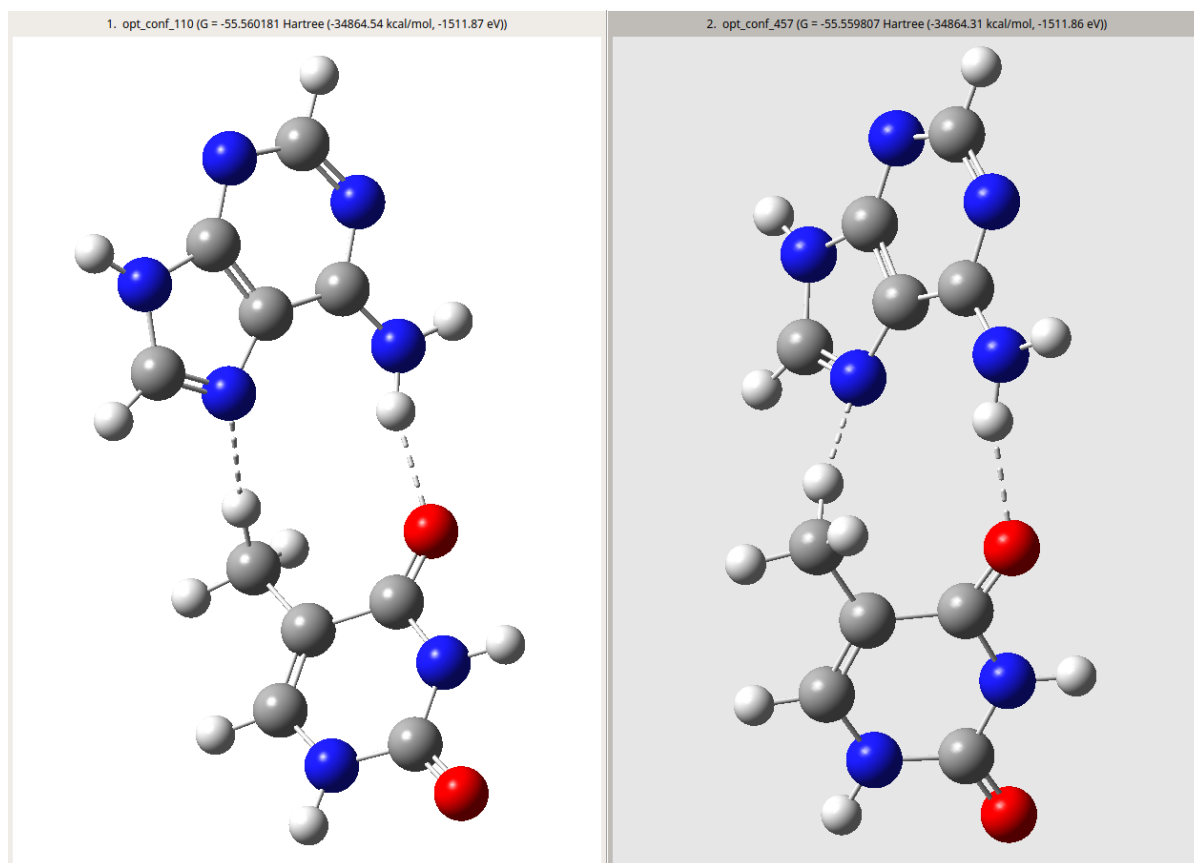


Figure 37: Two distinct spatial arrangements of the Adenine-Thymine dimer initially grouped into a single topological cluster.

By enabling the **RMSD Refinement (Step II)**, the algorithm successfully distinguished these isomers. The secondary geometric check decomposed **cluster_17** into two unique entities (**cluster_140** and **cluster_141**), despite their nearly identical Gibbs energies.

```
Cluster 140 (1 configurations) | RMSD Validated from Cluster 17:
Files:
- opt_conf_110.out (Gibbs Energy: -55.560181 Hartree (-34864.54 kcal/mol))

Cluster 141 (1 configurations) | RMSD Validated from Cluster 17:
Files:
- opt_conf_457.out (Gibbs Energy: -55.559807 Hartree (-34864.31 kcal/mol))
```

A pairwise comparison confirms the geometric distinctness, yielding a heavy-atom RMSD of $> 6.6 \text{ \AA}$, indicating a rotation of the complex that the scalar feature vector could not detect.

```
cosmic --compare opt_conf_110.out opt_conf_457.out
```

```
RMSD Analysis (Heavy Atoms):
Pairwise RMSD values between configurations:
opt_conf_110.out vs opt_conf_457.out: 6.655 \AA
```

14.3.2 Case Study B: Handling Symmetry and Indexing (Arsenate Anion)

While RMSD refinement offers geometric precision, it is computationally expensive and sensitive to atom indexing, which can lead to "over-clustering" in high-symmetry systems. This is illustrated by the monohydrated dihydrogen arsenate anion ($H_2AsO_4^- \cdot H_2O$) (Figure 38).

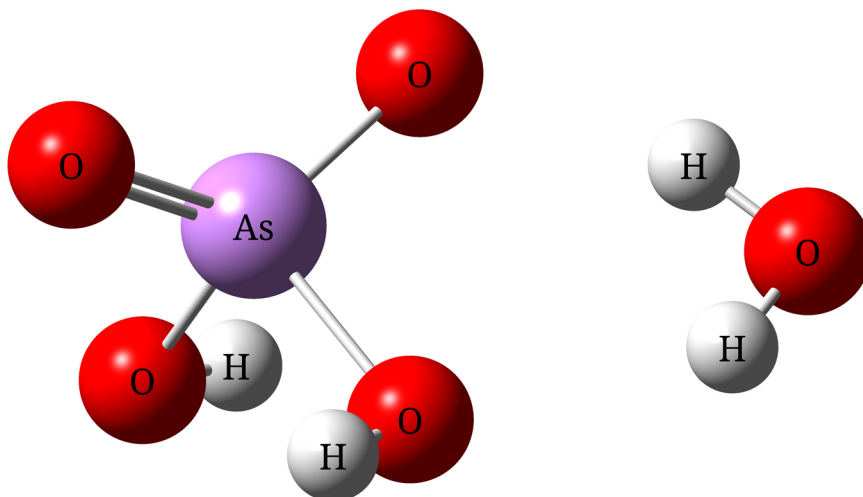


Figure 38: The monohydrated dihydrogen arsenate anion system.

The standard topological clustering (Step I) correctly identified a unified motif (`cluster_2_20`) containing 20 chemically identical structures (Figure 39).

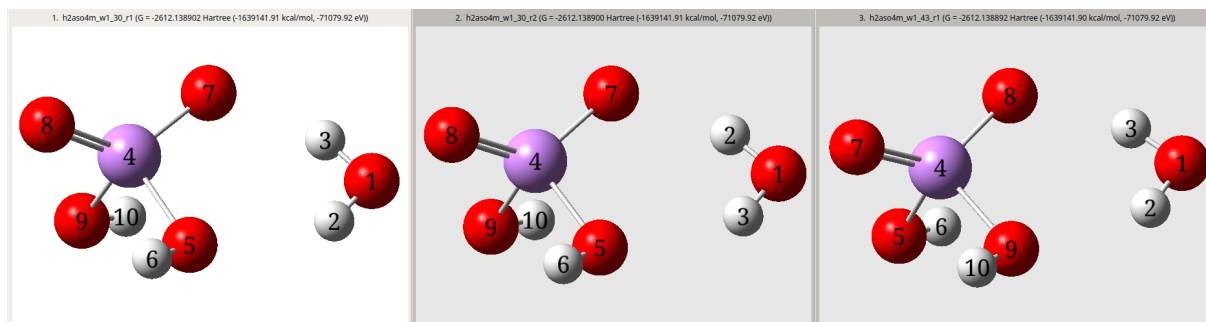


Figure 39: Correctly unified cluster identified by standard topological analysis.

However, applying RMSD refinement forced a split of this group. Due to atom indexing permutations inherent to the stochastic generation process, the RMSD algorithm flagged 8 structures as "distinct," creating a redundant artifact (`cluster_3_8`, Figure 40). These structures are chemically indistinguishable from the parent cluster.

Cluster 3 (8 configurations) | RMSD Validated from Cluster 2:

- h2aso4m_w1_12_r2.log (Gibbs Energy: -2612.138878 Hartree (-1639141.89 kcal/mol))
- h2aso4m_w1_38_r1.log (Gibbs Energy: -2612.138874 Hartree (-1639141.89 kcal/mol))

...

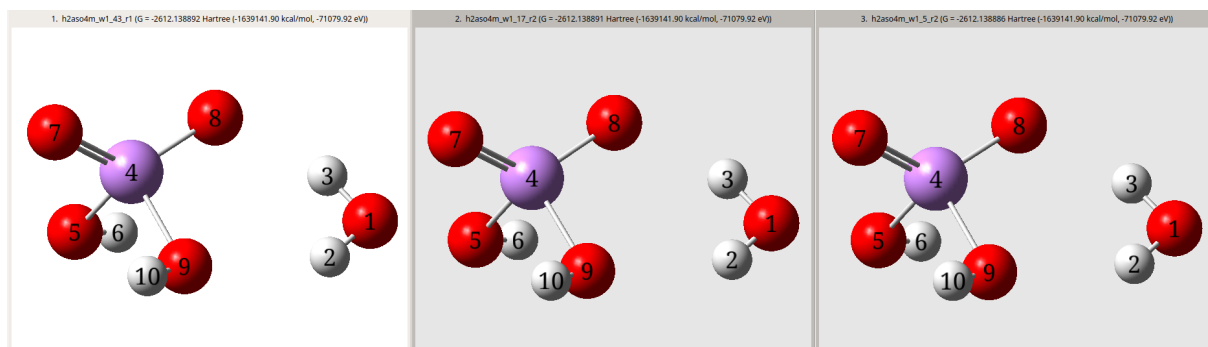


Figure 40: Redundant cluster artifact generated by RMSD sensitivity to atom indexing.

14.3.3 Clustering Overview

The RMSD refinement acts as a rigorous geometric filter, necessary for isolating specific stereoisomers (Case A). However, for general PES exploration, it risks inflating the ensemble size with false positives due to indexing permutations (Case B). Therefore, the ASCEC protocol defaults to topological vector clustering (Step I) for maximum efficiency, reserving RMSD (Step II) as an optional flag for systems where specific spatial orientation is critical.

15 Command Reference

15.1 Sampling and Replication

Commands used to initiate Simulated Annealing runs.

```
# Basic Execution
ascec input.asc           # Single run

# Replication (rN)
ascec input.asc r3        # 3 Replicated runs (Default packing)
ascec input.asc r5 --box10 # 5 Replicated runs with 10% packing density

# Box Validation
ascec input.asc box       # Analyzes simulation box requirements
```

15.2 Automated Workflow (Protocol)

Controls the execution of the multi-stage pipeline.

Command Format	Description
<code>... protocol</code>	Starts (or resumes) the automated workflow defined.
<code>... protocol N</code>	Force Restart. Restarts the pipeline from stage <i>N</i> (wiping progress for that stage).
<code>... protocol N -i</code>	Resume Interactive. Resumes stage <i>N</i> from where it left off (incomplete state).

Workflow: Annealing → Calculation → COSMIC → Optimization → COSMIC

15.3 Protocol thresholds

These commands generate input files for Quantum Chemistry packages (ORCA/Gaussian) and define the error handling logic.

Flag	Description
<code>--retry=N</code>	Execution Retries. Maximum attempts to restart a calculation that crashed or failed (e.g., walltime limit).
<code>--redo=N</code>	Topology Correction. Maximum attempts to fix imaginary frequencies by perturbing coordinates along the unstable mode.
<code>--critical=N</code>	Critical Threshold (Pre-opt). Max % of allowed "Critical" structures (isolated imaginary freqs). Set to 0 for strict filtering.
<code>--skipped=N</code>	Skipped Threshold (Final Opt). Max % of allowed "Skipped" structures. Set to 0 to ensure all final motifs are true minima.

15.4 COSMIC Analysis Module

Invoked via `ascec cosmic` or `cosmic`, this module handles clustering and Boltzmann analysis.

Flag	Description
<code>--th, --threshold</code>	COSMIC Threshold. Defines the clustering granularity. Default <code>auto</code> (knee detection, capped at $\tau = 2.0$; see Section 11.3.4); a float forces a fixed cut, lower values giving stricter clustering.
<code>--rmsd</code>	Geometric Validation. Enables RMSD checking in Ångströms (Default: 1.0 Å). Useful for distinguishing conformers with similar energies but different shapes.
<code>-j, --cores</code>	Parallelization. Number of CPU cores to use for parsing and matrix calculation (Default: Auto-detect).
<code>-T, --temperature</code>	Boltzmann Temp. Temperature in Kelvin for Gibbs Free Energy ranking (Default: 298.15 K).
<code>--weights</code>	Custom Weights. Define specific feature importance (e.g., <code>'(energy=0.8)(dipole=0.15)'</code>).
<code>--group-hb</code>	H-Bond Grouping. Groups structures by hydrogen-bond count before clustering. Produces separate dendrograms per HB family (<code>dendrogram_H{N}.png</code>). Useful for visual inspection of specific interaction networks.
<code>--reprocess-files</code>	Ignores the cached <code>.pk1</code> data and forces a re-read of all output.

16 Quick ORCA 6.1.1 Installation for Linux

1. Check for AVX2 Support

To determine if your Linux system's CPU supports the AVX2 instruction set for optimal performance, execute the following command in your terminal:

```
grep -i avx2 /proc/cpuinfo
```

If this command produces any output, your CPU supports AVX2. If the output is empty, you should use the non-AVX2 version of ORCA.

2. Download ORCA and OpenMPI

Register and log in to the official ORCA forum to download the appropriate files for ORCA 6.1.1. You will need to download both the ORCA package and the corresponding OpenMPI version.

Compounds search: <https://orcaforum.kofo.mpg.de/app.php/portal>

Compounds search: <https://www.open-mpi.org/>

```
/
├─ openmpi-4.1.8.tar.gz
├─ orca_6_1_1_linux_x86-64_shared_openmpi418.tar.xz
└─ orca_6_1_1_linux_x86-64_shared_openmpi418_avx2.tar.xz
```

3. Install OpenMPI

It is recommended to install the provided OpenMPI version before installing ORCA. Extract and compile the OpenMPI source code according to the instructions included with the download.

```
mkdir openmpi-4.1.8
```

```
tar -xvf openmpi-4.1.8.tar.gz -C openmpi-4.1.8/ --strip-components=1
```

```
cd openmpi-4.1.8
```

```
./configure --prefix=/home/user/software/openmpi-4.1.8
```

```
make -j4
```

It tells make to run 4 jobs simultaneously, you can use `make -j$(nproc)` instead.

```
make install
```

```
mpirun --version  
mpirun (Open MPI) 4.1.8
```

4. Install ORCA

Extract the downloaded ORCA tarball. For example:

```
mkdir orca_6_1_1
```

```
tar -xvf orca_6_1_1_linux_x86-64_shared_openmpi418.tar.xz -C orca_6_1_1 --strip-components=1
```

This will create a directory containing the ORCA executables.

5. Set Environment Variables

To make the ORCA program executable from any location in your terminal, you need to add its installation directory to your system's PATH. Follow these steps:

Open your shell's configuration file.

Open the appropriate configuration file in a terminal-based text editor like `micro`. If you are using the default Bash shell, use this command:

```
micro ~/.bashrc
```

Add the ORCA path to the file.

Note: The example directories for `orca` and `mpi` are located in `$HOME/software`.


```
# ORCA 6.1.1 Environment Configuration

# Define the paths to your ORCA 6.1.1 installation
export ORCA_BASE=$HOME/software
export ORCA611_ROOT=$ORCA_BASE/orca_6_1_1
export OPENMPI418_ROOT=$ORCA_BASE/openmpi-4.1.8

# ORCA and OpenMPI directories to the system's PATH
export PATH="$ORCA611_ROOT:$OPENMPI418_ROOT/bin:$PATH"

# ORCA and OpenMPI libraries to the library path
export LD_LIBRARY_PATH="$ORCA611_ROOT:$OPENMPI418_ROOT/lib:$LD_LIBRARY_PATH"
```

Save the file (Ctrl+S) and exit (Ctrl+Q). Then, reload the configuration:

Apply the changes.

For the changes to take effect in your current terminal session, you must "source" the configuration file.

```
source ~/.bashrc
```

Verify installation.

Be careful when having the Orca reader package installed. Orca is a screen reader pre-installed on most GNOME-based Linux distributions. To remove the Orca screen reader on Linux, use the terminal command.

```
sudo apt remove --purge orca
```

```
sudo apt autoremove
```

Then verify the ORCA 6.1.1 installation

```
orca --version
```

There is no standalone version command; however, the version (e.g., Program Version 6.1.1) is displayed at the beginning of the standard output for any run.

17 AI Acknowledgement and Reproducibility

The development and documentation of the ASCEC protocol involved the use of large language models (LLMs) to improve code optimization, data synthesis, and technical writing. Specifically, **Claude Opus 4.5** [16] and **Gemini 3.0** [17] were used to streamline the programming process and refine the algorithmic logic.

In accordance with ethical research standards, all AI-assisted results underwent rigorous human verification to ensure scientific accuracy and reproducibility. We encourage users who adapt this methodology to maintain transparency regarding the use of computational assistants and to verify all results.

18 References

- [1] P. J. van Laarhoven and E. H. Aarts. *Simulated Annealing: Theory and Applications*. Springer Science & Business Media, 1987.
- [2] N. Metropolis et al. “Equation of State Calculations by Fast Computing Machines”. In: *J. Chem. Phys.* 21.6 (1953). DOI: [10.1063/1.1699114](https://doi.org/10.1063/1.1699114).
- [3] J. F. Pérez, C. Z. Hadad, and A. Restrepo. “Structural Studies of the Water Tetramer”. In: *Int. J. Quantum Chem.* 108.10 (2008). DOI: [10.1002/qua.21615](https://doi.org/10.1002/qua.21615).
- [4] T. M. Abramyan et al. “Cluster Analysis of Molecular Simulation Trajectories for Systems Where Both Conformation and Orientation of the Sampled States Are Important”. In: *J. Comput. Chem.* 37.21 (2016). DOI: [10.1002/jcc.24416](https://doi.org/10.1002/jcc.24416). PMID: 27292100.
- [5] W. Kabsch. “A Solution for the Best Rotation to Relate Two Sets of Vectors”. In: *Acta Crystallogr. Sect. A* 32.5 (1976). DOI: [10.1107/S0567739476001873](https://doi.org/10.1107/S0567739476001873).
- [6] F. Neese. “Software Update: The ORCA Program System—Version 6.0”. In: *WIREs Comput. Mol. Sci.* 15.2 (2025). DOI: [10.1002/wcms.70019](https://doi.org/10.1002/wcms.70019).

- [7] C. Bannwarth, S. Ehlert, and S. Grimme. “GFN2-xTB—An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method with Multipole Electrostatics and Density-Dependent Dispersion Contributions”. In: *J. Chem. Theory Comput.* 15.3 (2019). DOI: [10.1021/acs.jctc.8b01176](https://doi.org/10.1021/acs.jctc.8b01176).
- [8] S. Kim et al. “An Update on PUG-REST: RESTful Interface for Programmatic Access to PubChem”. In: *Nucleic Acids Res.* 46.W1 (2018). DOI: [10.1093/nar/gky294](https://doi.org/10.1093/nar/gky294).
- [9] N. Rego and D. Koes. “3Dmol.js: Molecular Visualization with WebGL”. In: *Bioinformatics* 31.8 (2015). DOI: [10.1093/bioinformatics/btu829](https://doi.org/10.1093/bioinformatics/btu829). PMID: 25505090.
- [10] G. Hincapié et al. “Structural Studies of the Water Hexamer”. In: *J. Phys. Chem. A* 114.29 (2010). DOI: [10.1021/jp103683m](https://doi.org/10.1021/jp103683m).
- [11] R. R. Sokal and C. D. Michener. “A Statistical Method for Evaluating Systematic Relationships”. In: *Univ. Kans. Sci. Bull.* 38.22 (1958).
- [12] V. Satopaa et al. “Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior”. In: *Proc. 2011 31st Int. Conf. Distrib. Comput. Syst. Workshop. ICDCSW '11. USA: IEEE Computer Society, 2011*. DOI: [10.1109/ICDCSW.2011.20](https://doi.org/10.1109/ICDCSW.2011.20).
- [13] K. Pearson. “VII. Note on Regression and Inheritance in the Case of Two Parents”. In: *Proc. R. Soc.* 58.347-352 (1895). DOI: [10.1098/rsp1.1895.0041](https://doi.org/10.1098/rsp1.1895.0041).
- [14] R. Mojena. “Hierarchical Grouping Methods and Stopping Rules: An Evaluation*”. In: *Comput J* 20.4 (1977). DOI: [10.1093/comjnl/20.4.359](https://doi.org/10.1093/comjnl/20.4.359).
- [15] P. Farfán et al. “Dimers of Formic Acid: Structures, Stability, and Double Proton Transfer”. In: *J. Chem. Phys.* 147.4 (2017). DOI: [10.1063/1.4985880](https://doi.org/10.1063/1.4985880).
- [16] Anthropic. *System Card: Claude Opus 4.5*. <https://www.anthropic.com/claude-opus-4-5-system-card>. 2025.
- [17] Google DeepMind. *Gemini 3 Deep Think: Advancing Science, Research and Engineering*. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-deep-think/>. 2026.